

数字信号处理实现技术 实验指导书

电子科技大学

目录

实验规则	3
前 言	6
实验一 FIR 滤波器实验	7
1 实验原理	7
2 实验步骤	7
2.1 调用已有 fir 程序运行并绘制和观察波形	7
2.2 设计一个实例	12
实验三 汇编语言-卷积运算 conv	16
1 实验原理	16
2 实验步骤	16
步骤一 打开软件 Visual DSP++ Environment、Matlab	16
步骤二 打开工程文件 conv.dpj	16
步骤三 选择运行这个 Project 的 session	17
步骤四 利用 matlab 生成不同信号的数据文件	19
步骤五 修改程序	19
步骤六 利用 plot 功能观察输入、输出序列的波形	20
实验四 DFT	21
1 实验原理	21
2 实验步骤	22
2.1 调用已有 DFT 程序运行并绘制和观察波形	22
实验五 FFT	29
1 实验原理	29
2 实验步骤	32
步骤一 打开软件 Visual DSP++ Environment,Matlab	32
步骤二 打开工程 FFT_spectrum.dpj	32
步骤三 打开源文件 fft_spectrum.asm 并分析理解	33
步骤四 具体修改程序并运行	33

实验规则

为了维护正常的实验教学次序，提高实验课的教学质量，顺利的完成各项实验任务，确保人身、设备安全，特定制如下实验规则：

一、 实验前必须对每个实验所要求的预备知识要充分预习，另外还要求：

- 1、 认真阅读本实验指导书分析掌握本次实验的基本原理；
- 2、 完成各实验预习要求中制定的内容；
- 3、 熟悉实验任务

二、 实验时，认真、仔细的写出源程序，进行调试，有问题向指导老师举手提问；

三、 实验时注意观察，如发现有异常现象（电脑故障或仪器故障），必须及时报告指导老师，严禁私自乱动；

四、 实验过程中应仔细观察实验数据并加以记录

五、 自觉保持实验室的肃静、整洁；实验结束后，必须清理实验桌，将实验设备、工具、连线按规定放好，并填写仪器设备使用记录。

六、 凡有下列情况之一者，不准做实验：

- 1、 实验开始后迟到 10 分钟以上者；
- 2、 实验中不遵守实验室有关规定，不爱护仪器，表现不好而又不服从管理教育者；

七、 实验后，必须认真做好实验报告，下次实验时交实验指导老师批阅。没有交实验报告者，在规定时间内没有完成视为缺做一

次实验。

八、 一次未做实验，本实验课成绩视为不及格，原则上与下一届学生进行重修。以上实验规则，请同学们自觉遵守，并互相监督。

前 言

“数字信号处理实现技术实验室”位于科 C508, 拥有 30 套 ADSP21061EZ-KIT、60 套 ADSPBF533/535EZ-KIT、60 台计算机和其他配套设备, 能同时开设 60 个学生的 DSP 应用技术实验。

实验室面向电子工程学院各专业方向的学生开设 4 个实验: VisualDSP++ 开发过程学习、基于 SIMULATOR 的卷积、DFT 以及 FFT 运算、FIR 滤波器设计实验。

通过一系列的 DSP 实验, 可以加深学生对 DSP 开发流程和相关技术的理解掌握, 突出我校人才培养的专业特色, 对提高我学院各专业方向学生的培养质量、突出培养特色能起到很好的推动作用。

本实验指导书为有效开展 DSP 技术实验提供指导和依据, 首先介绍基于本实验室可开展的功能实验, 然后分节介绍每个电子实验的实验目的、实验内容及实验步骤。

实验一 FIR 滤波器实验

1 实验原理

FIR（即 Finite Impulse Response：有限冲激响应）滤波器在数字通信系统中被大量使用，以实现各种各样的功能，诸如低通滤波、带通滤波、抗混叠、抽样和内插等等。对于一个 FIR 滤波器系统而言，它的冲激响应总是有限长的，最基本的 FIR 滤波器可用下式表示：

$$y(n) = \sum_{k=0}^{N-1} h(k)x(n-k)$$

其中 $x(n)$ 是输入采样序列， $h(i)$ 是滤波器系数， N 是滤波器的系数长度， $y(n)$ 表示滤波器的输出序列。

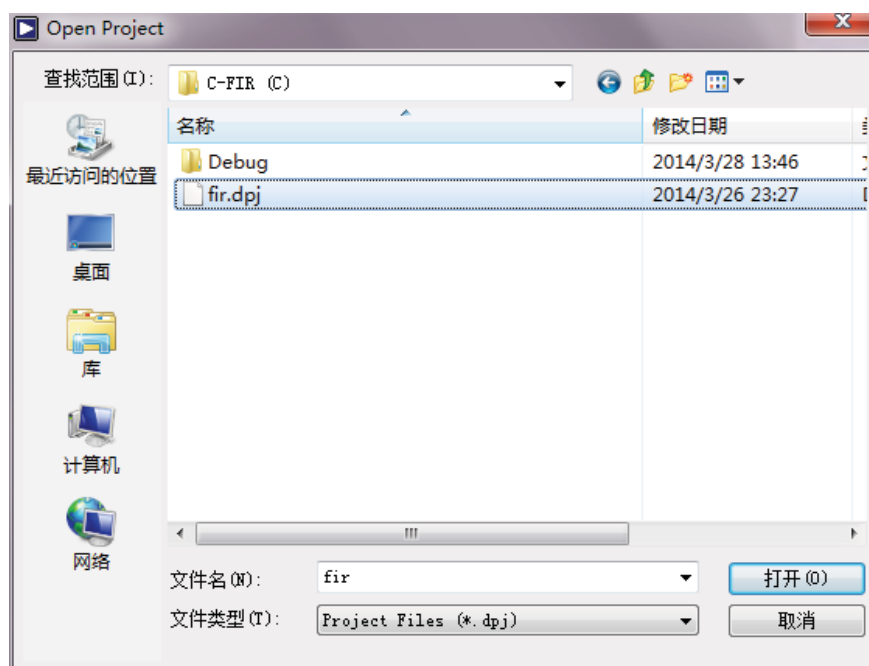
2 实验步骤

1. 调用已有 FIR 程序并学会调试运行，绘制和观察运行得到的波形图。
2. 设计一个由两个频率叠加的信号，通过 FIR 滤波器之后滤掉高频部分。

2.1 调用已有 fir 程序运行并绘制和观察波形

步骤一 打开 Visual DSP++ 并打开自带例子中的 fir 项目

具体步骤：File—Open—Project，打开 Open Project 窗口，选择安装路径下的 Analog Devices/VisualDSP 5.0/21k/Examples/No Hardware Required/C-FIR(C)/fir.dpj，



选择并点击打开：
得到上图。

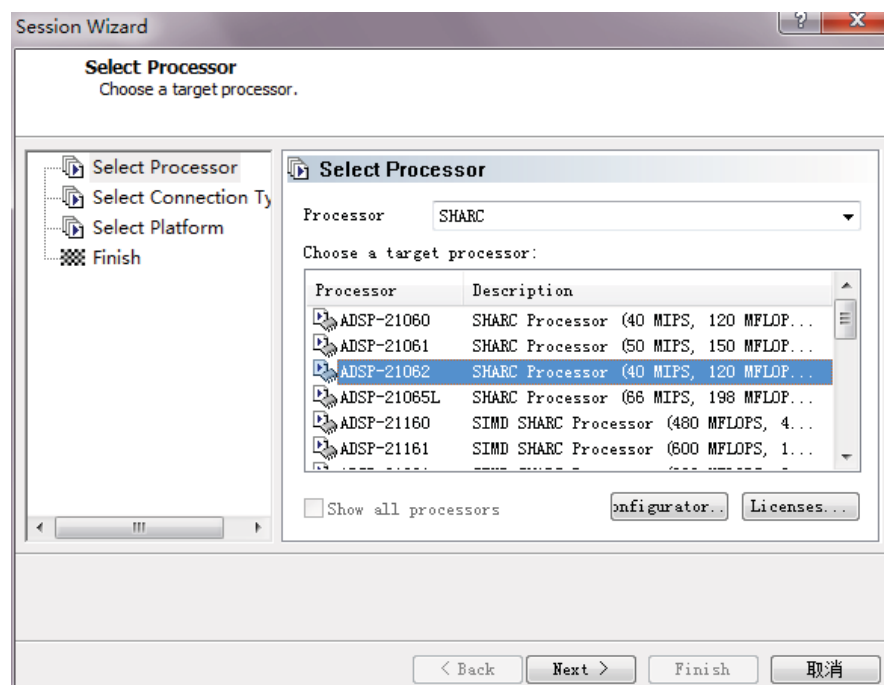
步骤二 选择运行这个 Project 的 session

首先查看目前的 Session，是否是我们需要的 ADSP-21062（在标题栏可以看到）。若不是就进行以下操作：

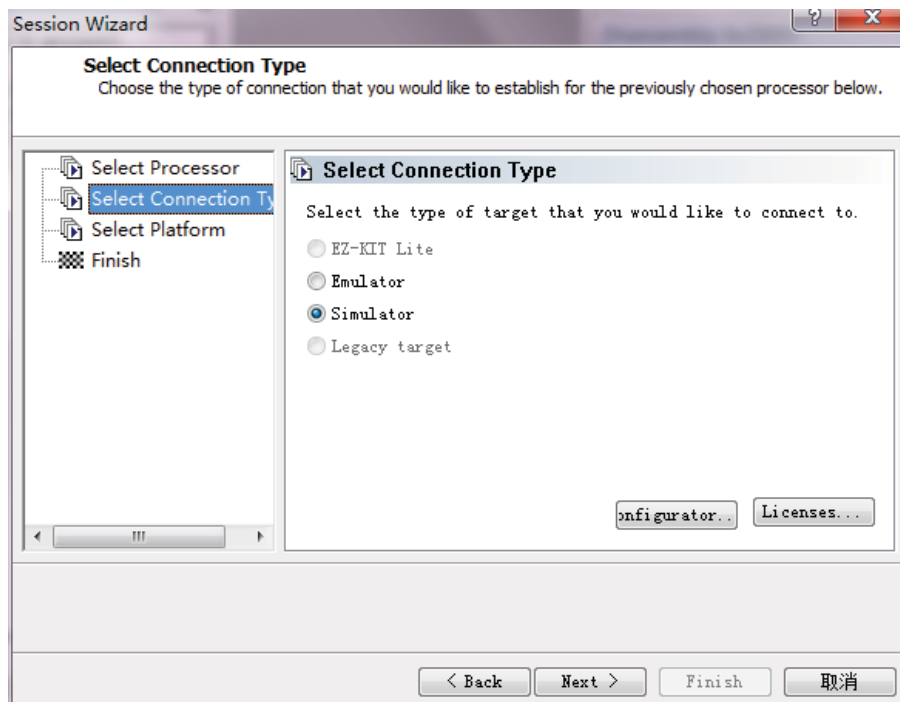
单击菜单栏上面的 Session-Select Session-ADSP-21062 ADSP-2106x Simulator。
若 Select Sessionz 中没有加载，就重新加载这个 Session，操作步骤如下。

单击菜单栏上面的 Session—New Session 进入选择界面 Session Wizard。

单击 Session Processor，在 Processor 中选择 SHARC，在 Choose a target processor 中选择 ADSP—21062，然后单击 Next。如下图：



单击 Next 之后出现 Select Session Type，选择 Simulator，再单击 Next:



以后两个界面单击 Next，Finish。

步骤三 调试 fir 程序

单击 Project—Build Project。

运行 fir 程序。单击 Debug—Run

步骤四 建立波形显示窗口

单击 View—Debug Windows—Plot—New，调出 Plot Configuration，设置 output 和 input。

Input 参数设置如下：

Title: 设置为程序的名称，此处是 FIR

Name: 设置为需要显示的变量名称，此处是 Input

Memory: 选择需要显示变量所在的地址区，此处是 Data(DM)Memory

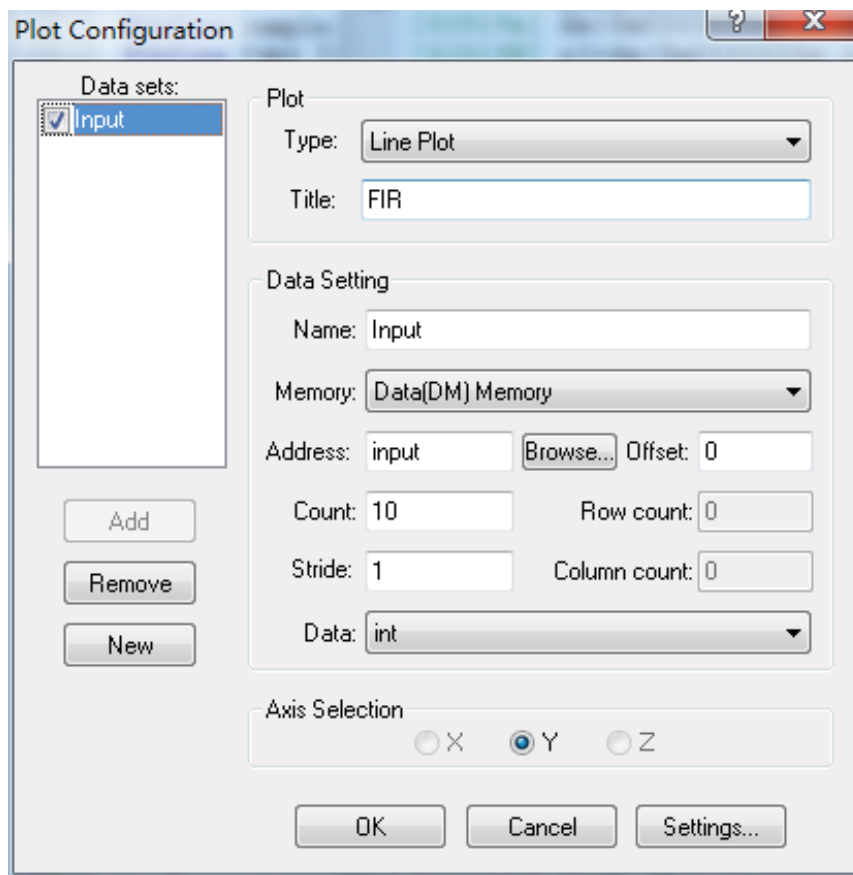
Address: 点击 Browse，选择要显示的变量，此处是 input

Count: 设置为要显示变量的点数，此处是 10

Stride: 显示的步长，此处设为 1

Data: 选择变量的类型，此处是 int

其他参数不变，点击 Add 添加，参数如下图：



Output 参数设置如下：

Title: 设置为程序的名称，此处是 FIR

Name: 设置为需要显示的变量名称，此处是 Output

Memory: 选择需要显示变量所在的地址区，此处是 Data(DM)Memory

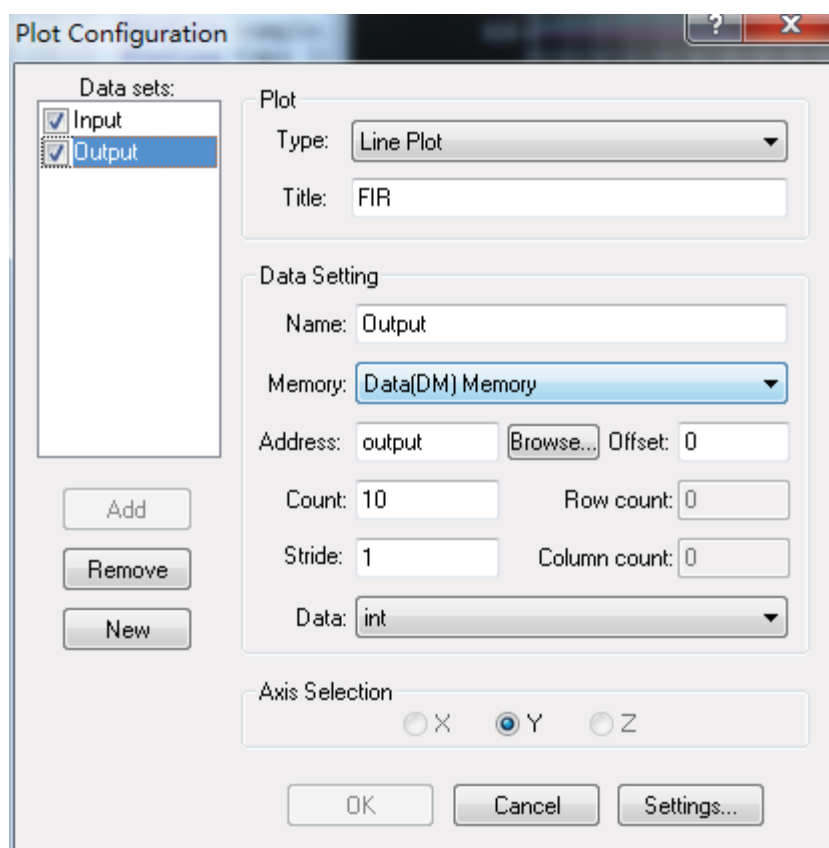
Address: 点击 Browse，选择要显示的变量，此处是 output

Count: 设置为要显示变量的点数，此处是 10

Stride: 显示的步长，此处设为 1

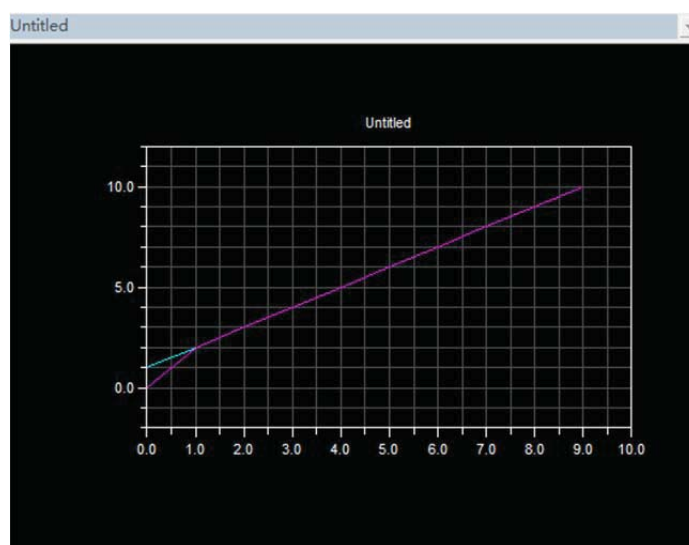
Data: 选择变量的类型，此处是 int

其他参数不变，点击 Add 添加，参数如下图



绘图设置好了以后可以在显示图像上点击右键选择 **Configure...**，可以修改已经设定的变量参数，也可以添加需要观察的变量。

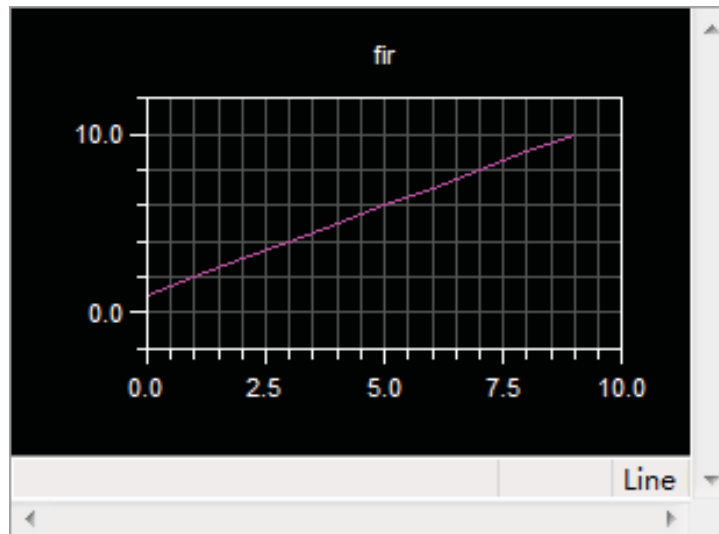
按照步骤三再重新调试和运行一下程序，得到波形图像如下：



步骤五：修改并调试程序。

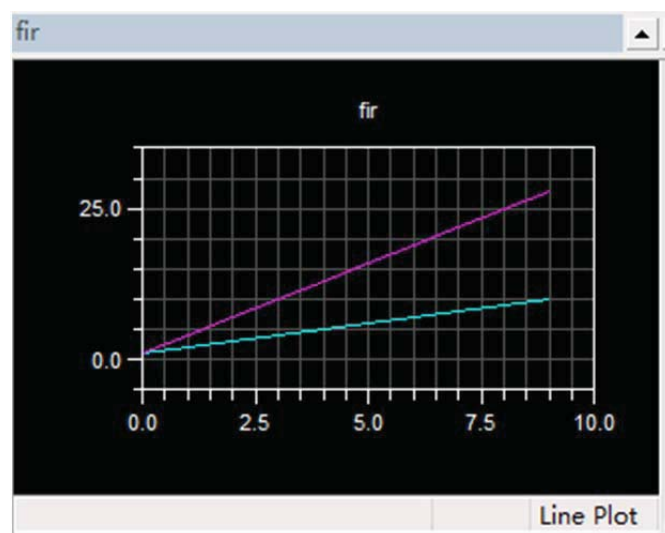
程序中的第三个 **for** 循环中的 **j > taps** 修改为 **j > -1**。

再次调试并运行程序，得到波形图像如下：



步骤六：自己修改数据并观察波形。在程序中修改 `coefficients` 的序列。观察波形图像。

比如将序列修改为{1,2,0,0,0,0,0,0,0,0,0,0,0,0,0}, 图像如下:



2.2 设计一个实例

步骤一

输入信号 $y = \sin(x_1) + \sin(x_2) = \sin(2\pi f_1 t) + \sin(2\pi f_2 t)$ ，其中 $f_1 = 4000\text{Hz}$ ， $f_2 = 2000\text{Hz}$ ，采样频率 $f_s = 10000\text{Hz}$ ，通过 matlab 以 $N = 32$ 采样得到信号的各采样点数据。

Matlab 程序如下:

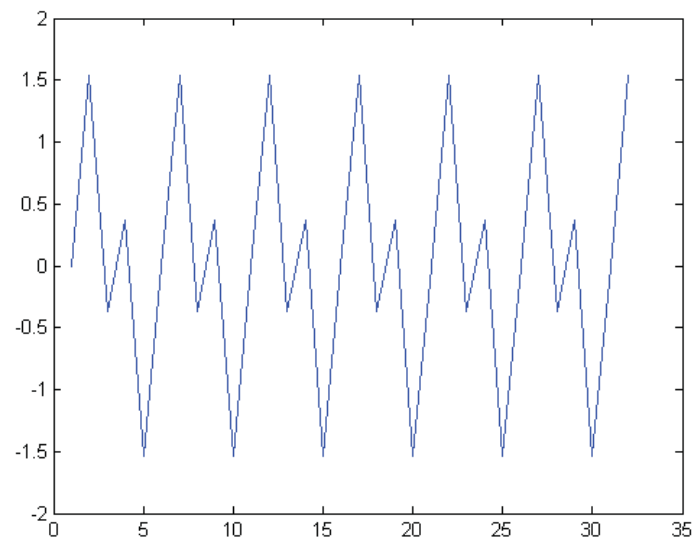
```
f1=4000
f2=2000;
fs=10000;
```

```

N=32
x1=sin(2*pi*f1*(0:N-1)/fs1);
x2=sin(2*pi*f2*(0:N-1)/fs1);
y1=x1+x2
fid = fopen('sin.txt','wt');
fprintf(fid,'%g\n',y1);
figure(1);
plot(y1);
Num;
y2=filter(Num,1,y1);
figure(2);
plot(y2);

```

得到输入信号图像如下：



得到输入信号的32个点数据如下：

```

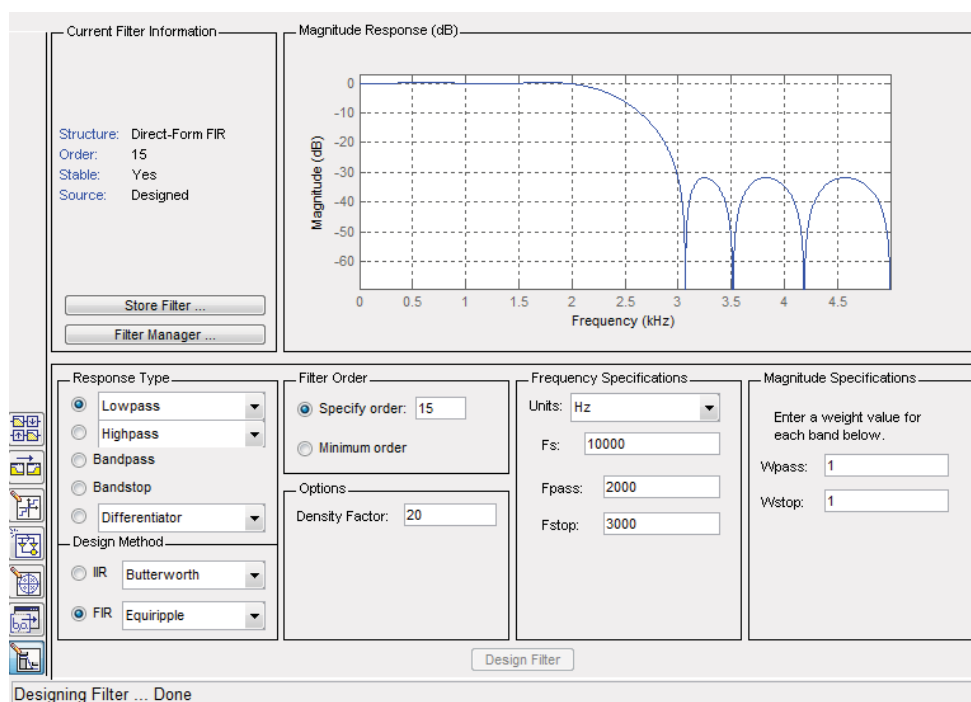
Y=0,0.3008,1.6276,-0.0000,1.3268,-1.3268,-0.0000,-1.6276,-0.3008,-0.0000,0.3008,1.6276,-0.0000,1.3268,-1.3268,0.0000,-1.6276,-0.3008,-0.0000,0.3008,1.6276,0.0000,1.3268,-1.3268,-0.0000,-1.6276,-0.3008,0.0000,0.3008,1.6276,0.0000,1.3268

```

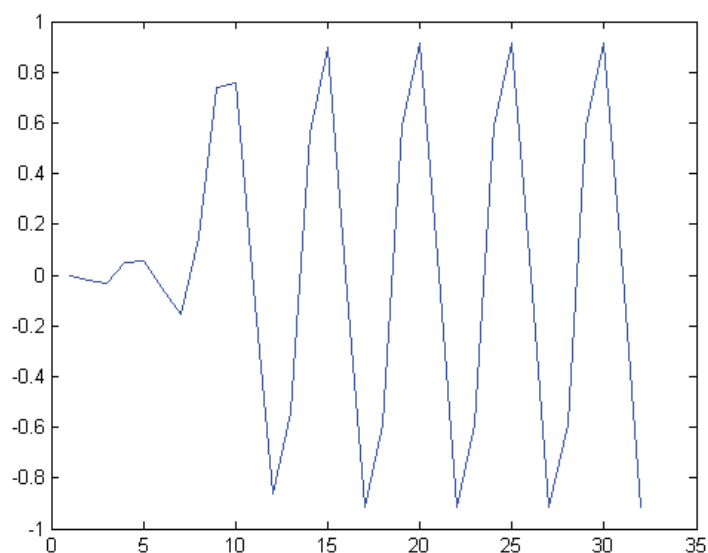
步骤二 在 matlab 中用 FDATOOL 工具设计一个低通滤波器

滤波器参数设置如下：

16 阶，Fs=10000HZ，Fpass=2000，Fstop=3000



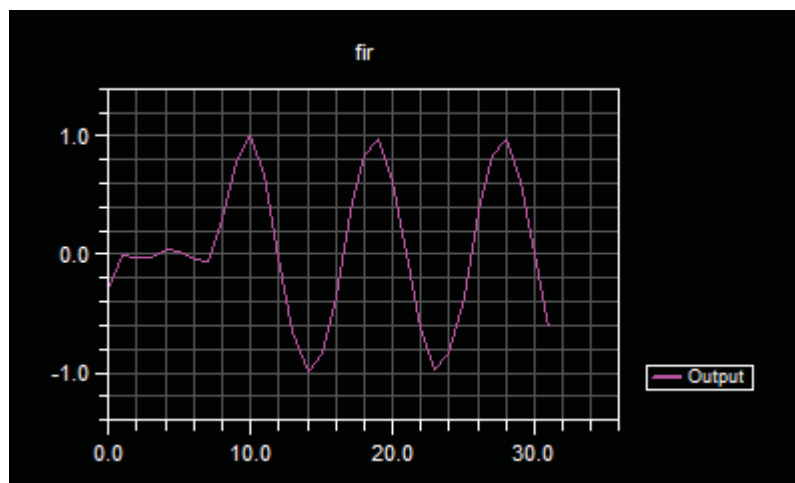
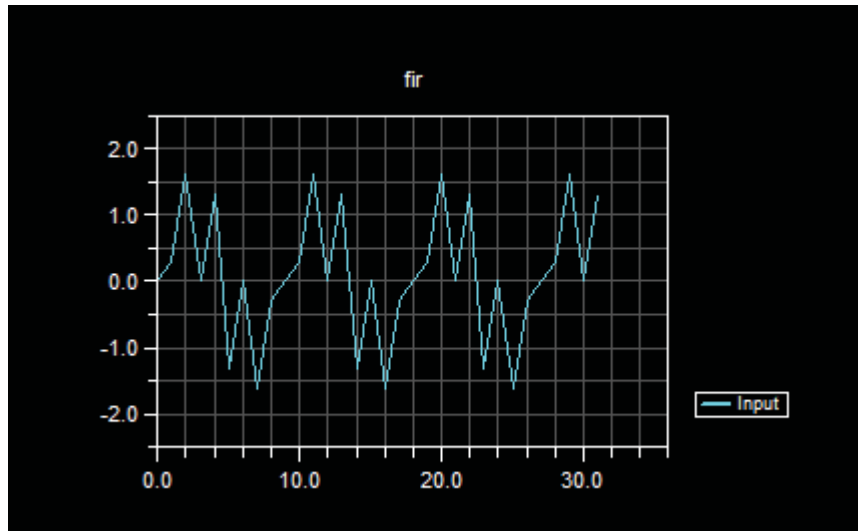
将所设计的滤波器系数导出到dat文件中，通过修改格式，得到如下系数：
-0.011726497289877891,-0.023618985632438541,0.028889445667084984,0.035581772905988421,-0.053943624122238618,-0.081984832920758963,0.14597295903772575,0.44799506238380371,0.44799506238380371,0.14597295903772575,-0.081984832920758963,-0.053943624122238618,0.035581772905988421,0.028889445667084984,-0.023618985632438541,-0.011726497289877891
同时将设计的滤波器导出到matlab中，对信号y进行滤波，得到滤波之后的信号如下图：



步骤三 将 matlab 中生成的数据添加到 VDSP 的

分别赋给输入信号 `input` 和滤波器系数 `coefficients`，同时修改 `input`，`output`，`coefficients`，`state`，还有中间变量 `temp` 的类型都改为 `float`，并修改其中数据的形式。

重复实验内容 1 中的步骤，运行之后得到 `input` 和 `output` 的图形如下



与 matlab 中生成的信号图形可知，已经对信号成功实现 fir 滤波，实验完成。可自行修改输入信号和滤波器系数，观察不同的实验结果。

实验三 汇编语言-卷积运算 conv

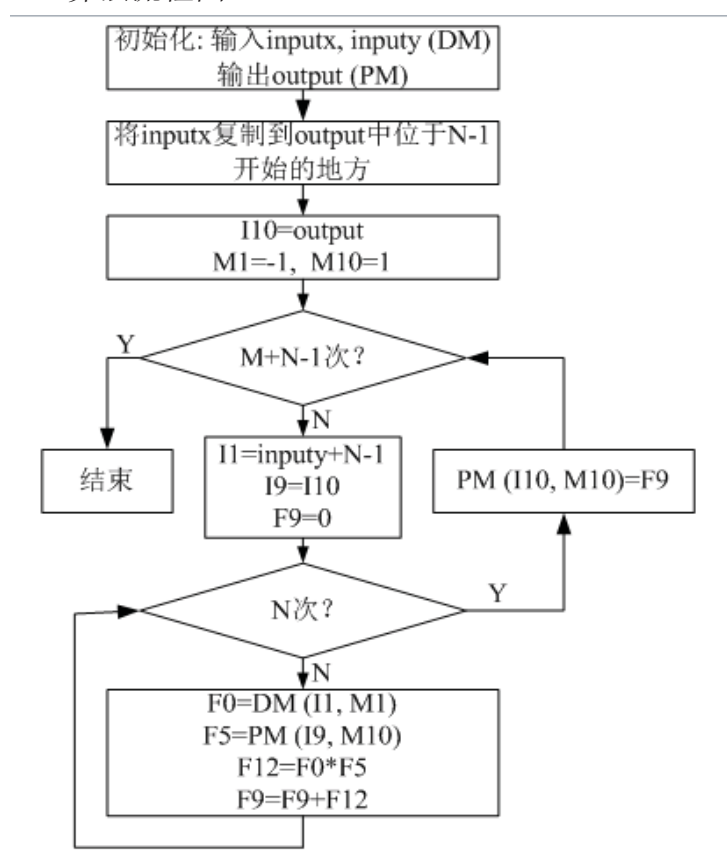
1 实验原理

1、输入序列：x 长度为 M，y 长度为 N

2、输出序列：z 长度为 M+N-1

3、公式：
$$z[m] = \sum_{k=0}^{M-1} x[k]y[m-k], \quad 0 \leq m \leq M+N-1$$

4、算法流程图



2 实验步骤

步骤一 打开软件 **Visual DSP++ Environment、Matlab**

步骤二 打开工程文件 **conv.dpj**

(本机工程文件路径为 D:\dsp\conv\Conv) 在工程管理窗口中打开源文件 Conv.asm，分析、理解源程序。

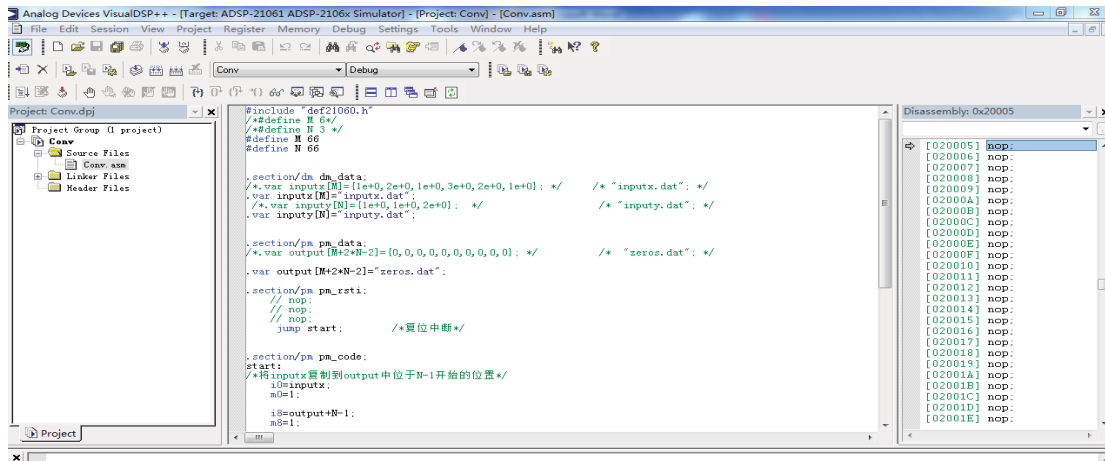


图 3.1 Conv.dpj 汇编源程序界面

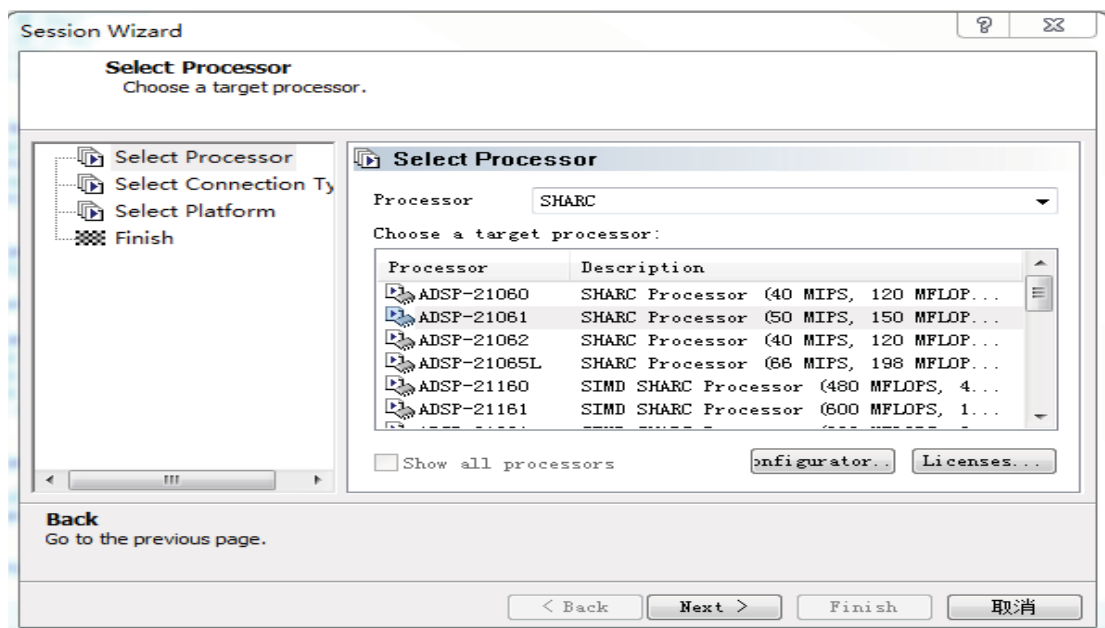
步骤三 选择运行这个 Project 的 session

首先查看目前的 Session，是否是我们需要的 ADSP-21061(主机不同，需要的 session 不同，若 ADSP-21061 不可用，即选择 ADSP-21060)（在标题栏可以看到）。若不是就进行以下操作：

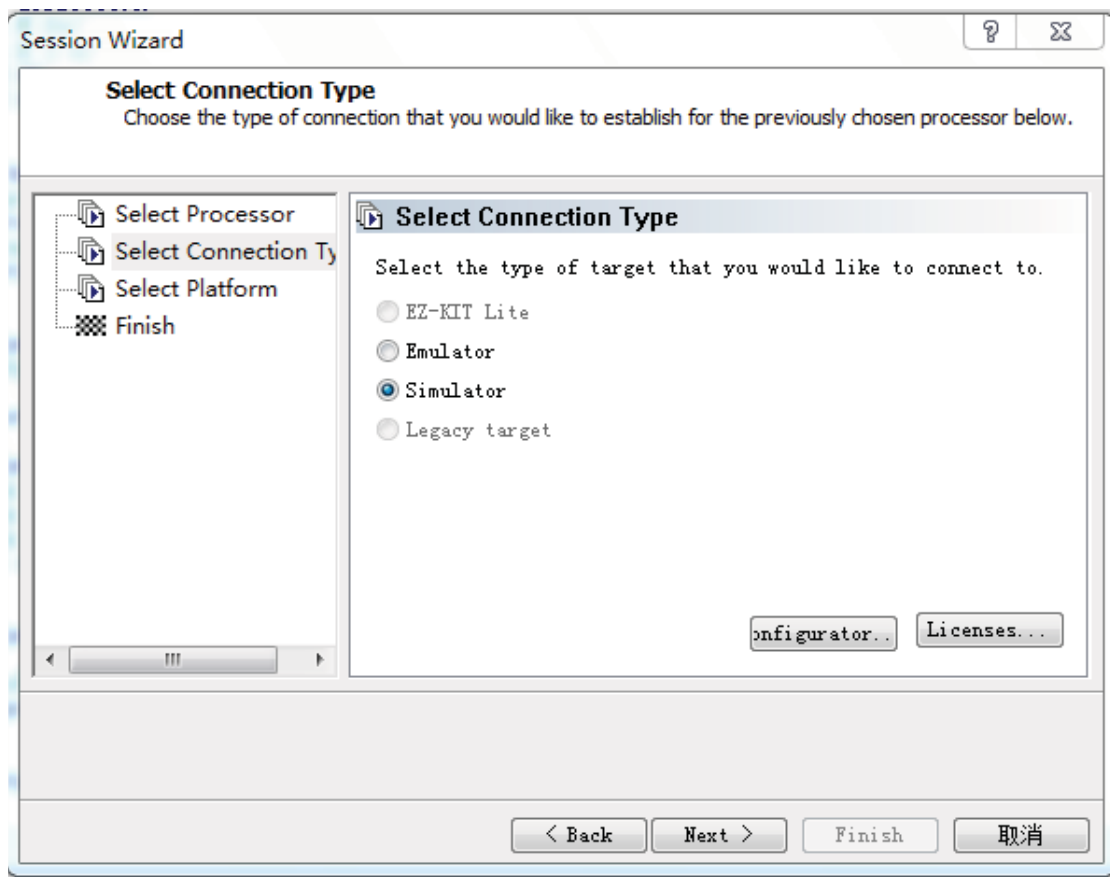
单击菜单栏上面的 Session->Select Session->ADSP-21062 ADSP-.2106x Simulator。若 Select Session 中没有加载，就重新加载这个 Session，操作步骤如下。

单击菜单栏上面的 Session—New Session 进入选择界面 Session Wizard。

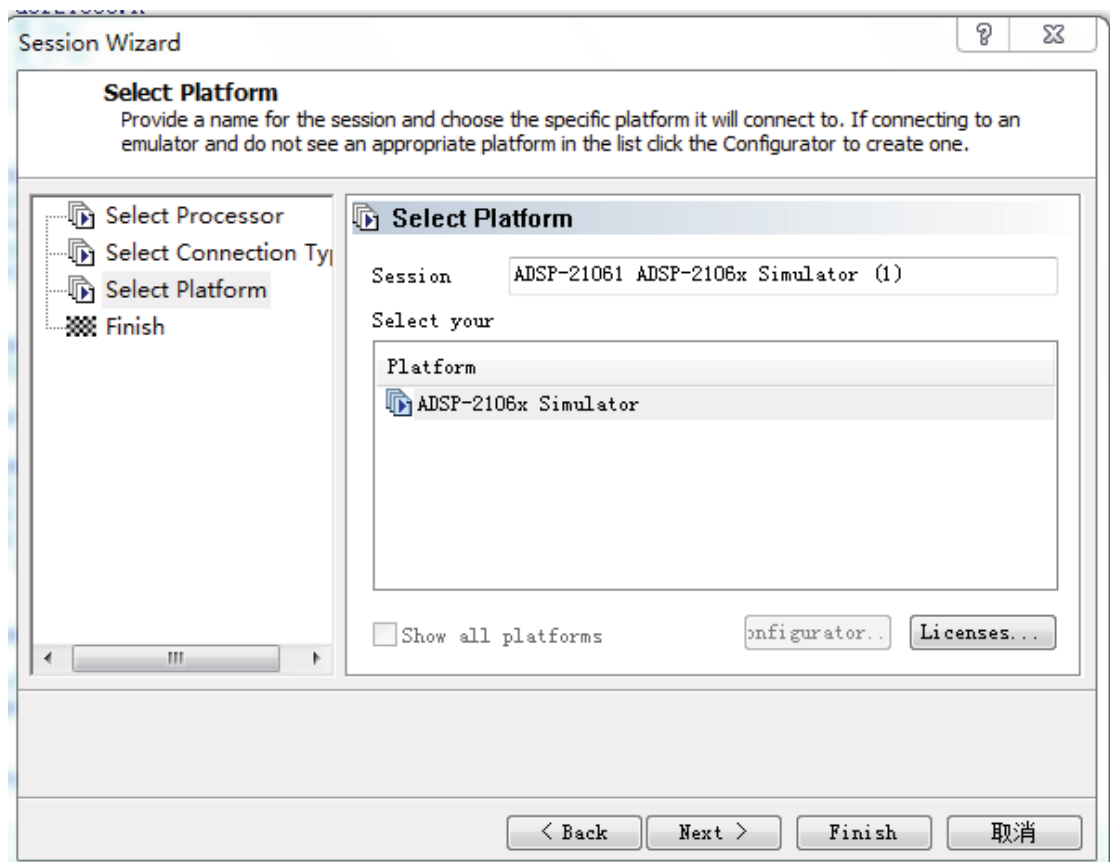
单击 Session Processor，在 Processor 中选择 SHARC，在 Choose a target processor 中选择 ADSP—21062。如下图：



然后单击 Next，如图：



点击 next，如图：



点出 Next->Finish，或直接 Finish.

步骤四 利用 matlab 生成不同信号的数据文件

包括：脉冲信号 tripulse.dat，正弦信号 sin1.dat，sin2.dat 方波信号 square1.dat，square2.dat，以及初始化文件 zeros.dat。本教程以长度为 66 点的三角波与自身做卷积为例，故 $M=N=66$ ，初始化文件 zeros.dat 长度为 $M+2*N-2=196$

数据文件生成程序为：（.dat 文件生成目录必须与工程文件目录相同，本机为 D:\dsp\conv\Conv，不同主机使用时注意修改路径）

```
clear all;
close all;
clc;

M=66;
N=66;

%%% pulse
x1=tripuls(-33:32,10);
z8=conv(x1,x1); %tripulse与自身卷积

%%% 画图
figure(1)
plot(x1); grid on;
title('tripulse');
figure(2)
plot(z8); grid on
title('convlution of tripulse andd tripulse');

%%% 写出tripulse.dat及zeros.dat文件
fid = fopen('D:\dsp\conv\Conv\tripulse.dat','w');
fprintf(fid,'%15.10e\n',x1);

z0=zeros(1,M+2*N-2);
fid = fopen('D:\dsp\conv\Conv\zeros.dat','w');
fprintf(fid,'%15.10e\n',z0);
fclose('all');
```

步骤五 修改程序

令输入序列x和y皆为tripulse.dat，输出序列初始化为zeros.dat，数据长度作相应修改（注：修改后需重新Build Project），如图：

```

/*#define M 6*/
/*#define N 3 */
#define M 66
#define N 66

.section/dm dm_data:
/*var inputx[M]={1e+0,2e+0,1e+0,3e+0,2e+0,1e+0};*/      /* "inputx.dat": */
var inputx[M]="tripulse.dat";
/*var inputy[N]={1e+0,1e+0,2e+0};*/                      /* "inputy.dat": */
var inputy[N]="tripulse.dat";

```

步骤六 利用 plot 功能观察输入、输出序列的波形

输入、卷积输出序列的波形如图 3.2、和图 3.3 所示：

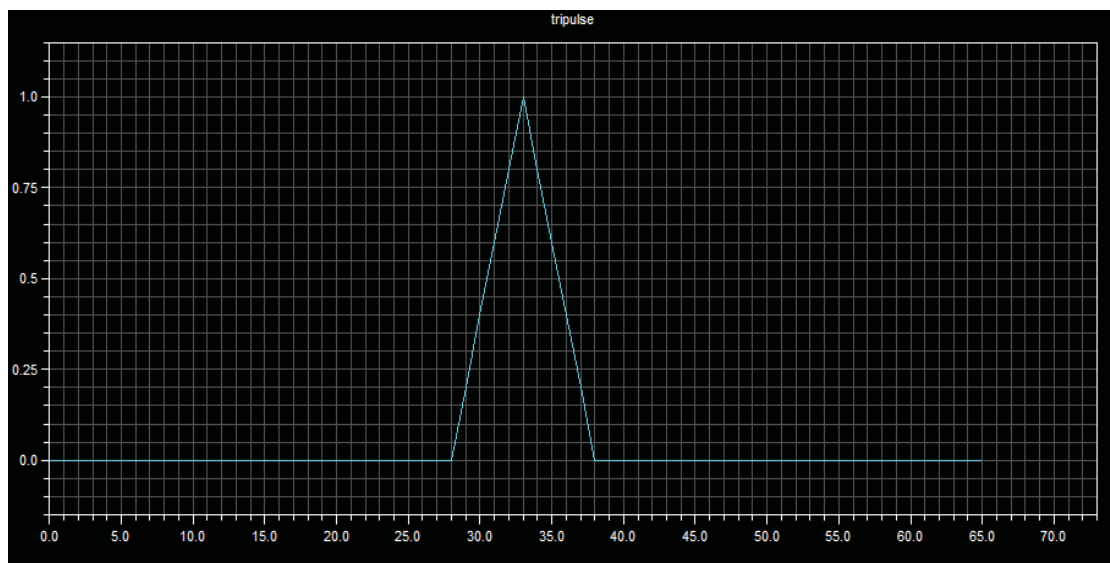


图 3.2 输入三角波波形

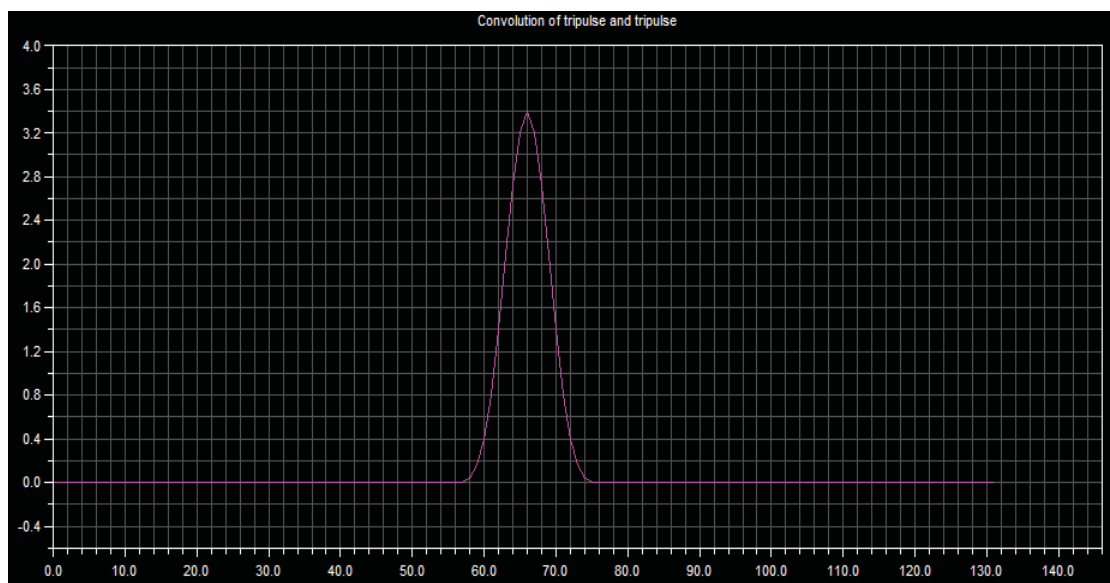


图 3.3 三角波与自身卷积波形

实验四 DFT

1 实验原理

DFT 的定义是针对任意的离散序列 $x(nT_s)$ 中的有限个离散抽样 ($0 \leq n < N$) 的, 它并不要求该序列具有周期性。

由 DFT 求出的离散谱 $H(k) = H_k = H\left(\frac{k}{NT_s}\right)$ ($k \in Z$) 是离散的周期函数, 周期为 $Nf_0 = N/T_0 = \frac{N}{NT_s} = \frac{1}{T_s} = f_s$ 、离散间隔为 $\frac{1}{NT_s} = \frac{f_s}{N} = \frac{1}{T_0} = f_0$ 。离散谱关于变元 k 的周期为 N 。

如果称离散谱经过 IDFT 所得到的序列为重建信号, $x'(nT_s)$ ($n \in Z$), 则重建信号是离散的周期函数, 周期为 $NT_s = T_0 = \frac{1}{f_0}$ (对应离散谱的离散间隔的倒数)、离散间隔为

$T_s = NT_s / N = \frac{T_0}{N} = \frac{1}{Nf_0}$ (对应离散谱周期的倒数)。

经 IDFT 重建信号的基频就是频域的离散间隔, 或时域周期的倒数, 为 $f_0 = \frac{1}{T_0} = \frac{1}{NT_s}$ 。实

序列的离散谱关于原点和 $\frac{N}{2}$ (如果 N 是偶数) 是共轭对称和幅度对称的。因此, 真正有用的频谱信息可以从 $0 \sim \frac{N}{2}-1$ 范围获得, 从低频到高频。在时域和频域 $0 \sim N$ 范围内的 N 点分别是各自的主值区间或主值周期。

DFT 性质:

线性性: 对任意常数 a_m ($1 \leq m \leq M$), 有 $DFT\left[\sum_{m=1}^M a_m x_m(n)\right] \Leftrightarrow \sum_{m=1}^M a_m DFT[x_m(n)]$

奇偶虚实性:

DFT 的反褶、平移: 先把有限长序列周期延拓, 再作相应反褶或平移, 最后取主值区间的序列作为最终结果。

DFT 有如下的奇偶虚实特性:

奇 $\Leftrightarrow$ 奇; 偶 $\Leftrightarrow$ 偶; 实偶 $\Leftrightarrow$ 实偶; 实奇 $\Leftrightarrow$ 虚奇;

实 $\Leftrightarrow$ (实偶) + j(实奇); 虚 $\Leftrightarrow$ (实偶)·EXP(实奇)。

反褶和共轭性:

时域	频域
反褶	反褶
共轭	共轭 + 反褶
共轭 + 反褶	共轭

对偶性: $X(n) \Leftrightarrow Nx(-k)$

把离散谱序列当成时域序列进行 DFT, 结果是原时域序列反褶的 N 倍; 如果原序列具有偶对称性, 则 DFT 结果是原时域序列的 N 倍。

时移性: $x(n-m) \Leftrightarrow X(k)W_N^{km}$ 。序列的时移不影响 DFT 离散谱的幅度。

频移性: $x(n)W_N^{-nl} \Leftrightarrow X(k-l)$

时域离散圆卷积定理: $x(n) \otimes y(n) \Leftrightarrow X(k)Y(k)$

圆卷积: 周期均为 N 的序列 $x(n)$ 与 $y(n)$ 之间的圆卷积为

$$x(n) \otimes y(n) = \sum_{i=0}^{N-1} x(i)y(n-i)$$

$x(n) \otimes y(n)$ 仍是 n 的序列, 周期为 N 。

非周期序列之间只可能存在线卷积, 不存在圆卷积; 周期序列之间存在圆卷积, 但不存在线卷积。

频域离散圆卷积定理: $x(n)y(n) \Leftrightarrow \frac{1}{N} X(k) \otimes Y(k)$

时域离散圆相关定理: $R_{xy}^{(P)}(n) \Leftrightarrow X(k)Y^*(k)$

周期为 N 的序列 $x(n)$ 和 $y(n)$ 的圆相关: $R^{(P)}(x(n), y(n)) \stackrel{\Delta}{=} R_{xy}^{(P)}(n) = \sum_{i=0}^{N-1} x(i)y^*(i-n)$

是 n 的序列, 周期为 N 。

$h(n) = \frac{1}{N} \left\{ DFT_k [H^*(k)] \right\}^*$ 。其中 $DFT_k[\cdot]$ 表示按 k 进行 DFT 运算。

帕斯瓦尔定理: $\sum_{n=0}^{N-1} \|x(n)\|^2 = \frac{1}{N} \sum_{k=0}^{N-1} \|X(k)\|^2$

2 实验步骤

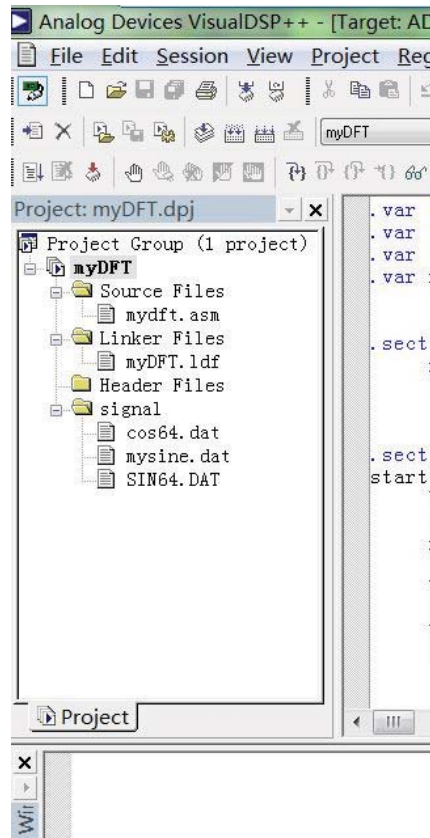
2、按要求设计正弦的输入信号, 通过 DFT 变换观察生成的波形。

2.1 调用已有 DFT 程序运行并绘制和观察波形

步骤一 打开工程

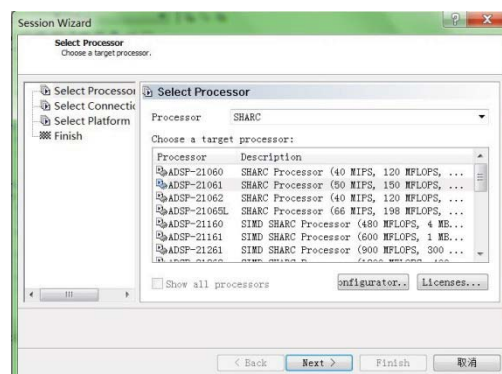
打开 Visual DSP++ 并打开自带例子中的 myDFT 工程:

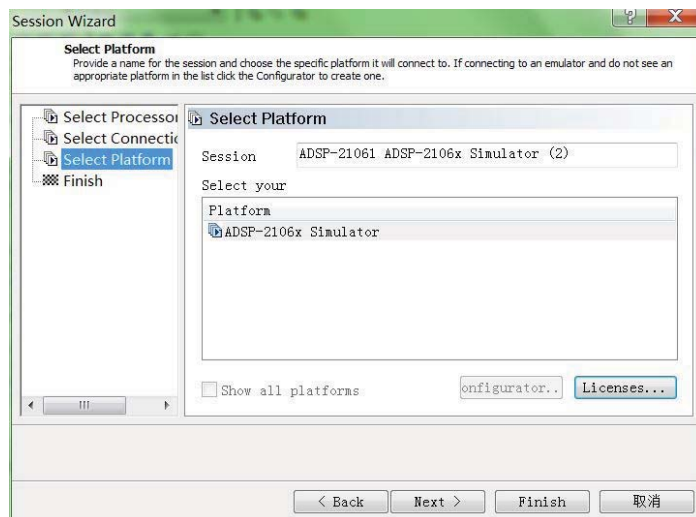
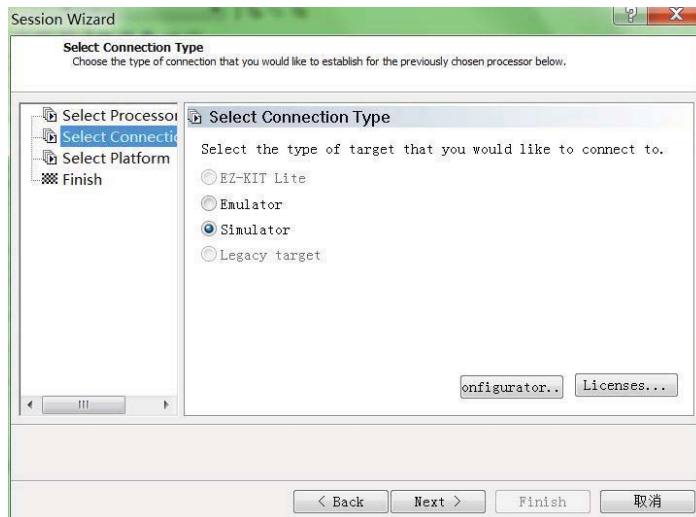
打开软件后, 加载工程:dsp\DFT\mydft\myDFT. dpj



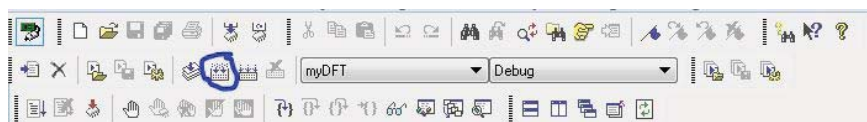
步骤二 新建 section:

在菜单栏单击 section，然后 new section。在 select processor 中的 processor 选项下选择 SHARC,然后 ADSP_21061, 本页设置结束点击 next, 在 select connection type 中选择 simulator 后点击 next, 最后点击 finish。



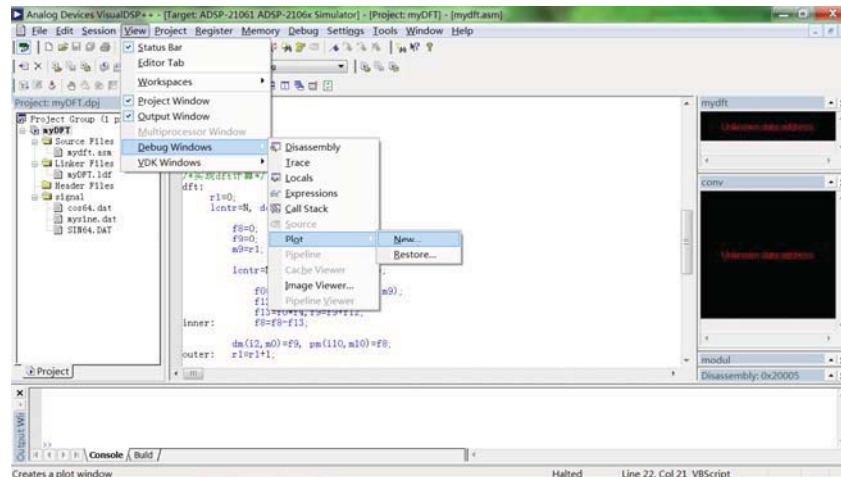


步骤三 编译



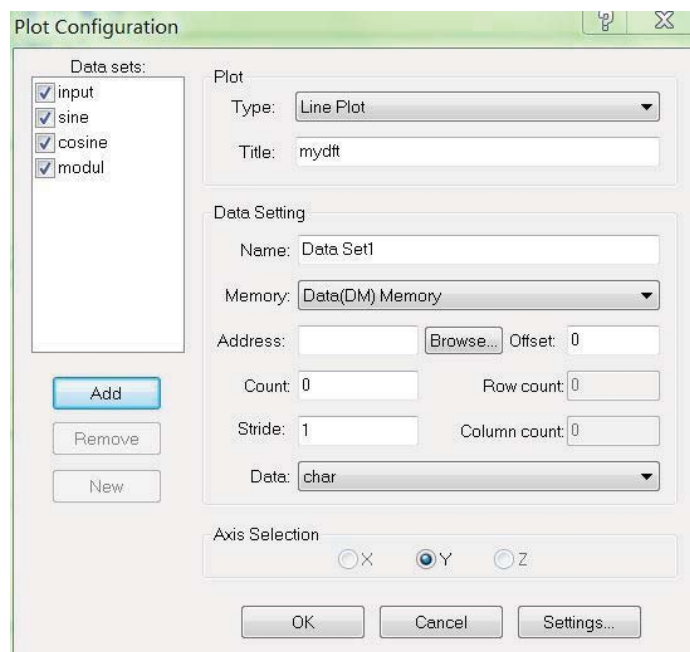
步骤四 打开画图窗口

在 view 菜单下，选择 debug windows 然后选择 plot，最后点击 new 打开配置对话框（plot configuration）。



步骤五 设置绘图

打开对话框之后先设置 plot 栏中的信息，plot 栏中的信息是只需要输入一次的。Plot 栏设置后就可以设置下面的 data setting 栏中的信息（信息如下表格），输入完第二列的信息之后 add，这就添加了第一个信号的信息。然后点击 new 设置二、三列，最后点击 ok。最后把对话框左上侧的 data sets 框中的信号选中（想看哪几个就选中哪几个）

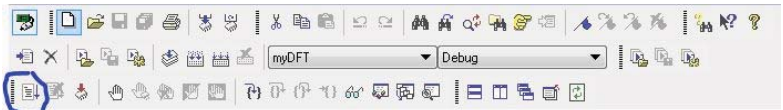


BOX	Input 信号	SIN 信号	COS 信号	取模后信号
Name	input	Sine	Cosine	Modul
Memory	Data(DM) Memory	Data(DM) Memory	Data(DM) Memory	Data(DM) Memory
Addresds	Input	Sine	Cosine	modul

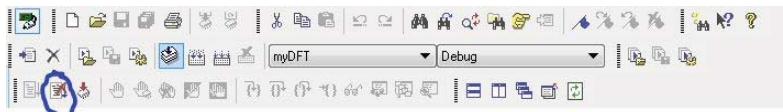
count	64	64	64	64
stride	1	1	1	1
Data	Short	short	short	short

步骤六 运行程序

点击 F5 或点击按钮

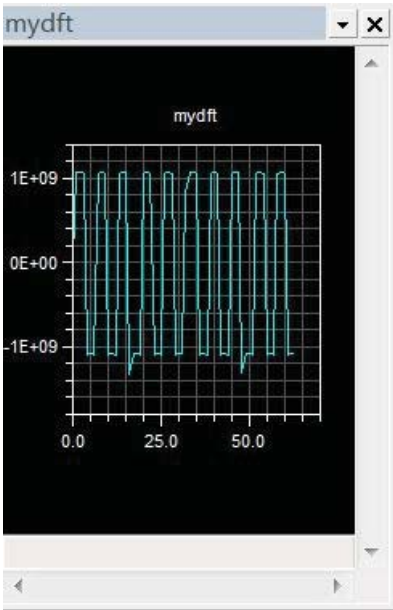


程序没有自动停下，再点击停止按钮

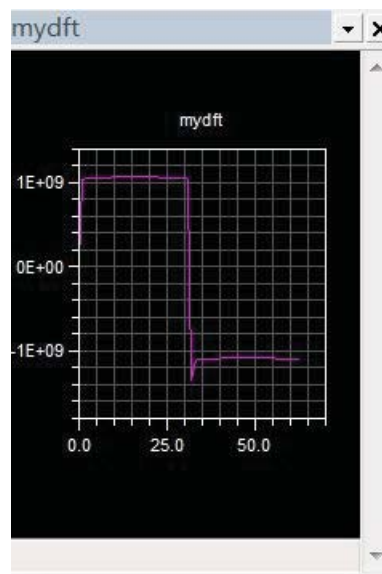


生成的图形如下图所示：

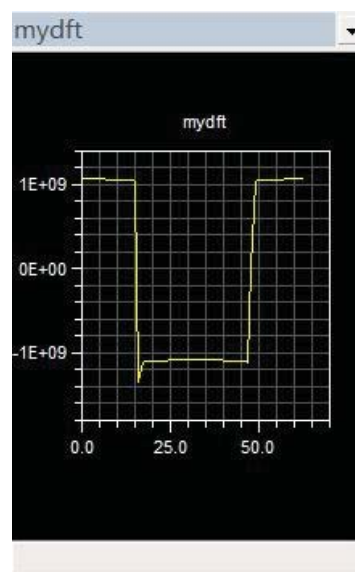
Input



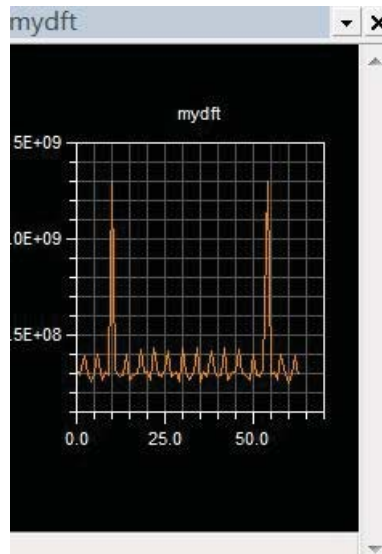
Sine



Cosine



Modul



步骤七 用 matlab 产生输入信号

用 matlab 打开文件……DFT\data_gen\ mysine1_dft.m
按要求修改信号的采样频率，信号频率等参数。保存后点击运行则会跳出信号的时域和频域图形，并产生信号文件 mysine1.dat
将这个文件复制到 mydft 工程文件所在目录下，再修改 visualdsp++ 的 mydft.asm 文件中的 var input[N]="mysine1.dat"; 的文件名，并与之对应。

步骤八 重复步骤一到步骤六完成内容二

实验五 FFT

1 实验原理

快速傅里叶变换 FFT

旋转因子 W_N 有如下的特性。

$$\text{对称性: } W_N^{k+N/2} = -W_N^k \quad (1)$$

$$\text{周期性: } W_N^{n(N-k)} = W_N^{k(N-n)} = W_N^{-nk} \quad (2)$$

利用这些特性,既可以使 DFT 中有些项合并,减少了乘法积项,又可以将长序列的 DFT 分解成几个短序列的 DFT。FFT 就是利用了旋转因子的对称性和周期性来减少运算量的。

FFT 的算法是将长序列的 DFT 分解成短序列的 DFT。例如: N 为偶数时,先将 N 点的 DFT 分解为两个 $N/2$ 点的 DFT,使复数乘法减少一半;再将每个 $N/2$ 点的 DFT 分解成 $N/4$ 点的 DFT,使复数乘又减少一半,继续进行分解可以大大减少计算量。最小变换的点数称为基数,对于基数为 2 的 FFT 算法,它的最小变换是 2 点 DFT。

一般而言,FFT 算法分为按时间抽取的 FFT (DIT FFT) 和按频率抽取的 FFT (DIF FFT) 两大类。DIF FFT 算法是在时域内将每一级输入序列依次按奇 / 偶分成 2 个短序列进行计算。而 DIT FFT 算法是在频域内将每一级输入序列依次奇 / 偶分成 2 个短序列进行计算。两者的区别是旋转因子出现的位置不同,得算法是一样的。在 DIF FFT 算法中,旋转因子 W_N 出现在输入端,而在 DIT FFT 算法中它出现在输出端。

假定序列 $x(n)$ 的点数 N 是 2 的幂,按照 DIF FFT 算法可将其分为偶序列和奇序列。

$$\text{偶序列: } x(2r) = x_1(r)$$

$$\text{奇序列: } x(2r+1) = x_2(r)$$

其中: $r=0, 1, 2, \dots, N/2-1$, 则 $x(n)$ 的 DFT 表示为

$$\begin{aligned} X(k) &= \sum_{n=0}^{N-1} x(n) W_N^{nk} = \sum_{n=0}^{N/2-1} x(n) W_N^{nk} + \sum_{n=0}^{N/2-1} x(n) W_N^{nk} \\ &= \sum_{r=0}^{N/2-1} x(2r) W_N^{2rk} + \sum_{r=0}^{N/2-1} x(2r+1) W_N^{(2r+1)k} \\ &= \sum_{r=0}^{N/2-1} x_1(r) (W_N^2)^{rk} + W_N^k \sum_{r=0}^{N/2-1} x_2(r) (W_N^2)^{rk} \end{aligned}$$

$$\begin{aligned}
&= \sum_{r=0}^{N/2-1} x_1(r) W_{N/2}^{rk} + W_N^k \sum_{r=0}^{N/2-1} x_2(r) W_{N/2}^{rk} \\
&= X_1(k) + W_N^k X_2(k) \quad r, k = 0, 1, \dots, N/2-1
\end{aligned}$$

式中, $X_1(k)$ 和 $X_2(k)$ 分别为 $X_1(r)$ 和 $X_2(r)$ 的 $N/2$ 的 DFT。由于对称性, $W_N^{k+N/2} = -W_N^k$ 。因此, N 点 DFT 可分为两部分:

$$\text{前半部分: } x(k) = x_1(k) + W_N^k x_2(k) \quad (3)$$

$$\text{后半部分: } x(N/2+k) = x_1(k) - W_N^k x_2(k) \quad k=0, 1, \dots, N/2-1 \quad (4)$$

从式 (4) 和式 (5) 可以看出, 只要求出 $0 \sim N/2-1$ 区间 $x_1(k)$ 和 $x_2(k)$ 的值, 就可求出 $0 \sim N-1$ 区间 $x(k)$ 的 N 点值。

以同样的方式进行抽取, 可以求得 $N/4$ 点的 DFT, 重复抽取过程, 就可以使 N 点的 DFT 用上组 2 点的 DFT 来计算, 这样就可以大减少运算量。

基 2 DIF FFT 的蝶形运算如下图所示。设蝶形输入为 $x_1(k)$ 和 $x_2(k)$, 输出为 $x(k)$ 和 $x(N/2+K)$, 则有

$$x(k) = x_1(k) + W_N^k x_2(k) \quad (5)$$

$$x(N/2+k) = x_1(k) - W_N^k x_2(k) \quad (6)$$

在基数为 2 的 FFT 中, 设 $N=2^M$, 共有 M 级运算, 每级有 $N/2$ 个 2 点 FFT 蝶形运算, 因此, N 点 FFT 总共有 $MN/2$ 个蝶形运算。

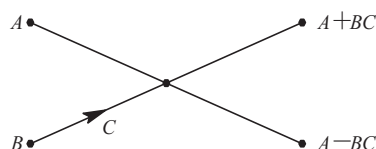


图 1 基 2 DIF FFT 的蝶形运算

例如: 基数为 2 的 FFT, 当 $N=8$ 时, 共需要 3 级, 12 个基 2 DIT FFT 的蝶形运算。其信号流程如图 2 所示。

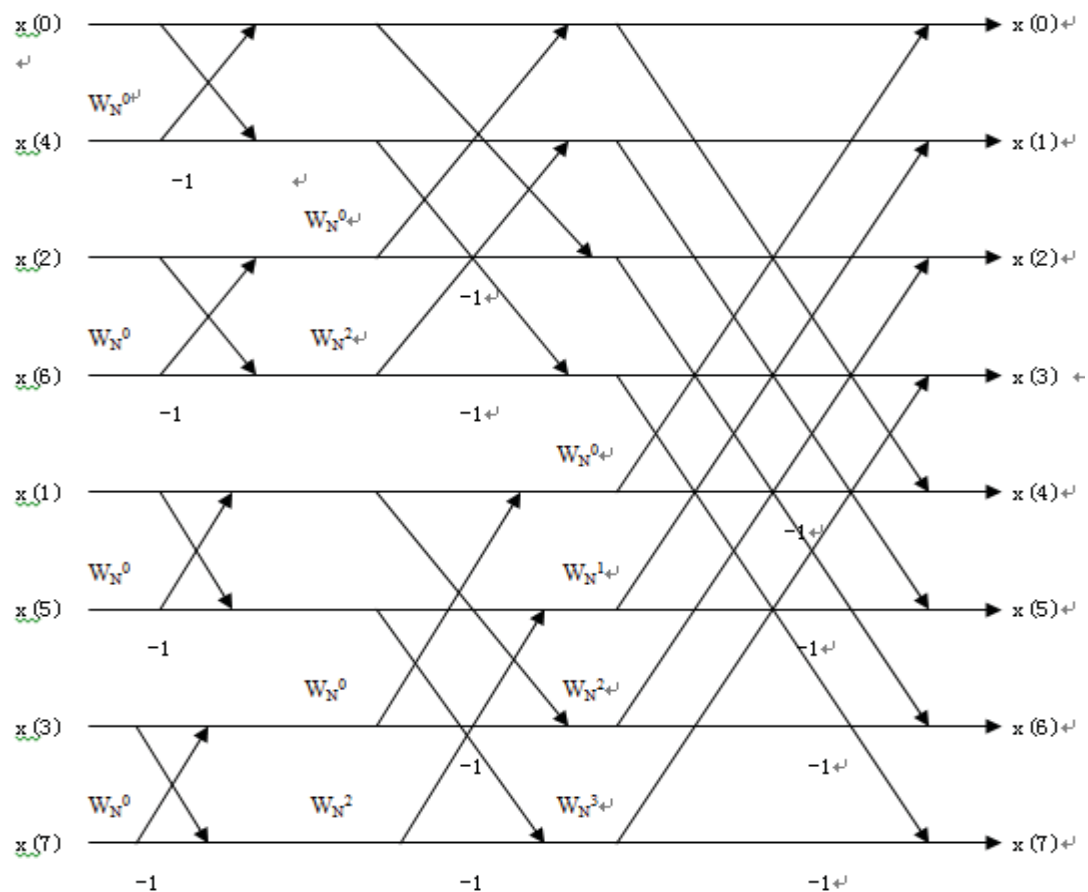
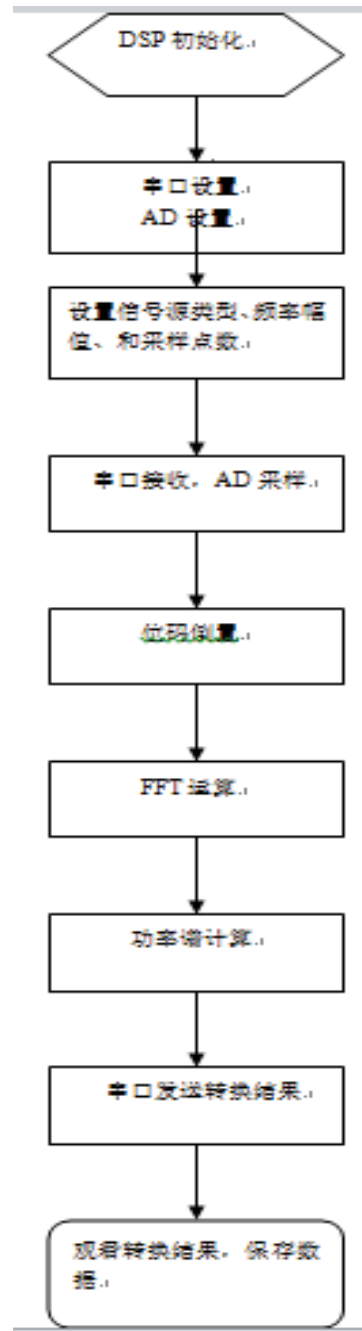


图 2 8 点基 2 DIF FFT 蝶形运算

从图(2)可以看出，输入是经过比特反转的倒位序列，称为位码倒置，其排列顺序为 $x(0), x(4), x(2), x(6), x(1), x(5), x(3), x(7)$ ，输出是按自然顺序排列，其顺序为 $x(0), x(1), x(2), x(3), x(4), x(5), x(6), x(7)$ 。

程序设计顺序



2 实验步骤

步骤一 打开软件 **Visual DSP++ Environment, Matlab**

步骤二 打开工程 **FFT_spectrum.dpj**

本机工程路径 D:\dsp\FFT\FFT_spectrum, 设置正确的 Session: ADSP-21060,

设置方法如上例所示。

步骤三打开源文件 `fft_spectrum.asm` 并分析理解

分析、理解源程序如下图所示：

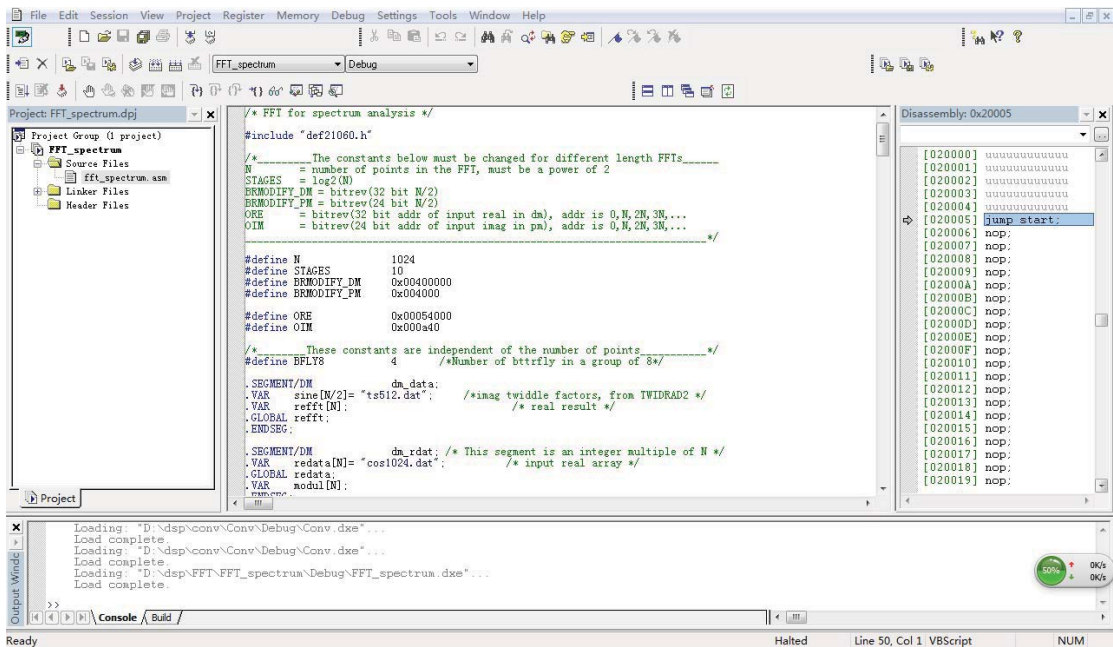


图 5.1 FFT_spetrum. dpj 汇编源程序界面

4、本实验具体要求为：

数据点数：1024 点

数据源：

采样频率：fs=80MHz:

(1) 信号频率：f=(30-1.23*分组号)MHz，正弦信号，注意视图坐标显示。

(2) 产生对应频率的复信号 $\cos(2\pi f t) + j\sin(2\pi f t)$ ，观测其频谱特征。

(3) 产生两个频率相邻的信号，观测并分析 fft 的分辨能力。

(4) 其他

步骤四 具体修改程序并运行

本题以第 8 组为例，故 f=20.16MHz

(1) 正弦信号：

a. 利用 matlab 生成数据文件正弦信号 `sin1024.dat`，与 `zeros1024.dat`，将其保存在与工程文件相同的目录下，并画出正弦信号时域波形与经 FFT 变换后的幅度响应，Matlab 程序为：

```
clear all;
```

```
close all;
```

```
clc
```

```
N=1024;
```

```
f0=20.16e6;
```

```

fs=80e6;
t=0:1/fs:1023/fs;
x0_sin=sin(2*pi*f0*t);
delta_1024=(-fs/2:fs/N:fs/2-fs/N)/1e6;

```

%%% 画出正弦信号时域波形

```

figure(1)
plot(t*1e6,x0_sin); grid on;
xlabel('时间/us');
ylabel('幅度');
title('正弦信号时域波形');

```

%%% 求正弦信号FFT变换并画出频谱响应

```

Xw=fft(x0_sin);
Xw=abs(Xw);
Xw=20*log10(Xw/max(Xw));
Xw=fftshift(Xw);
figure(2)
plot(delta_1024,Xw); grid on;
xlabel('频率/MHz');
ylabel('幅度/dB');
title('正弦信号经FFT变换幅度响应');

```

%%% 写出.dat文件（注意路径一定要与工程文件FFT_spectrum.dpj相同）

```

fid = fopen('D:\dsp\FFT\FFT_spectrum\sin1024.dat','w');
fprintf(fid,'%15.10e\n',x0_sin);

z0=zeros(1,N);
fid = fopen('D:\dsp\FFT\FFT_spectrum\zeros1024.dat','w');
fprintf(fid,'%15.10e\n',z0);

```

b.在 Visual DSP++中，只需令汇编程序中的输入实部为 sin1024.dat，输入虚部为 zeros1024.dat，即可，如下图所示，

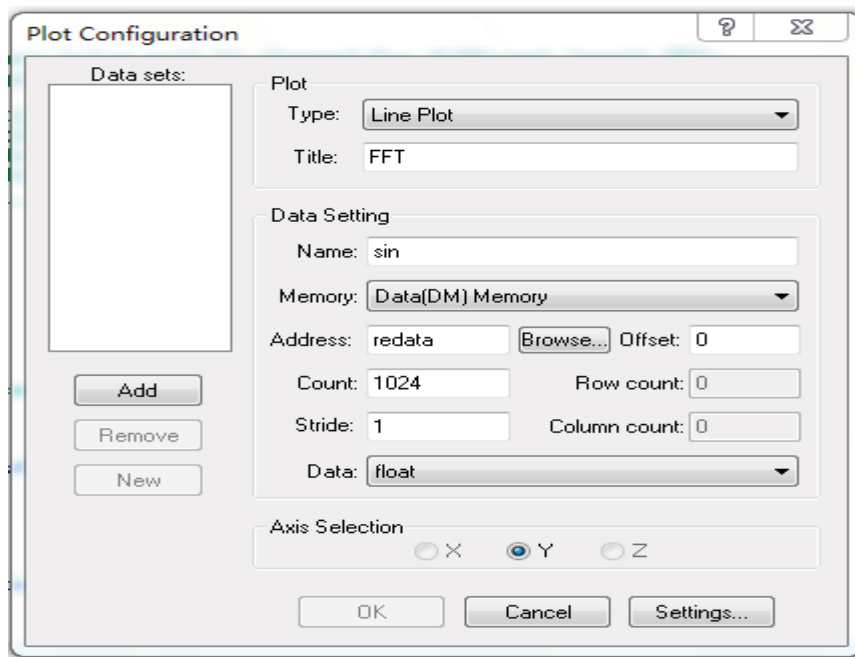
```

.VAR redata[N]= "sin1024.dat"; .VAR imdata[N]= "zeros1024.dat";

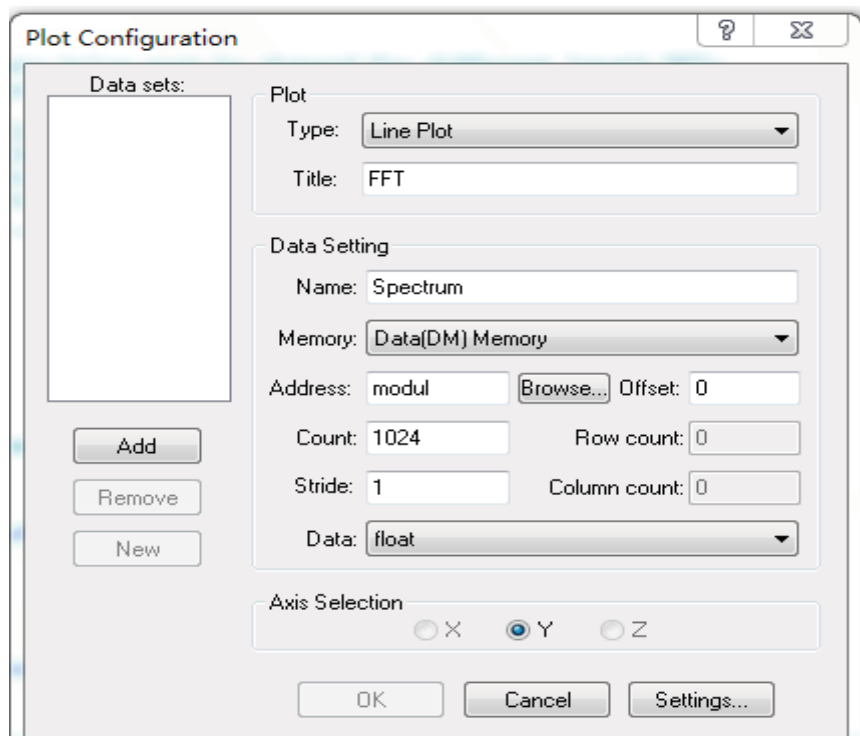
```

c. Build Project。

d. 新建图形，画出正弦信号时域波形：View->Debug Windows->Plot->New，在弹出的 Plot Configuration 中设置如下：



e. 另新建一图形，画出经 FFT 变换后频谱，设置如下：



f. 运行程序，可看到正弦信号的时域及频域波形

h. 正弦信号的理想与实际时域及频域波形如下图所示：

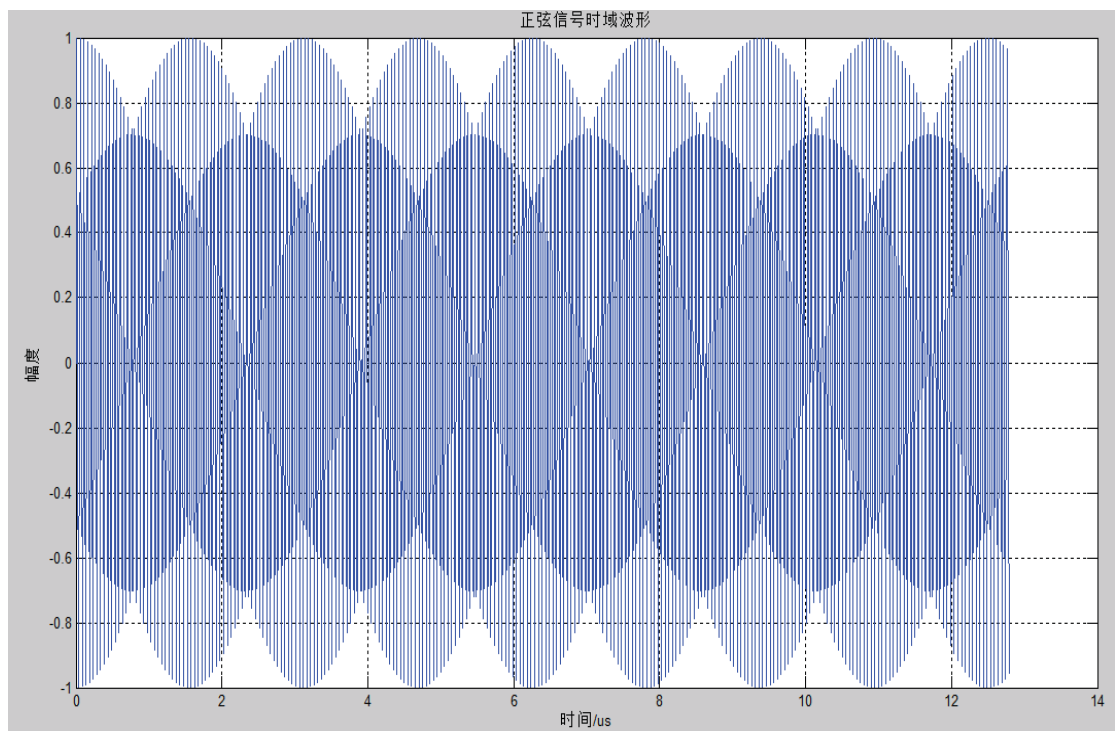


图 5.2 正弦信号的理想波形

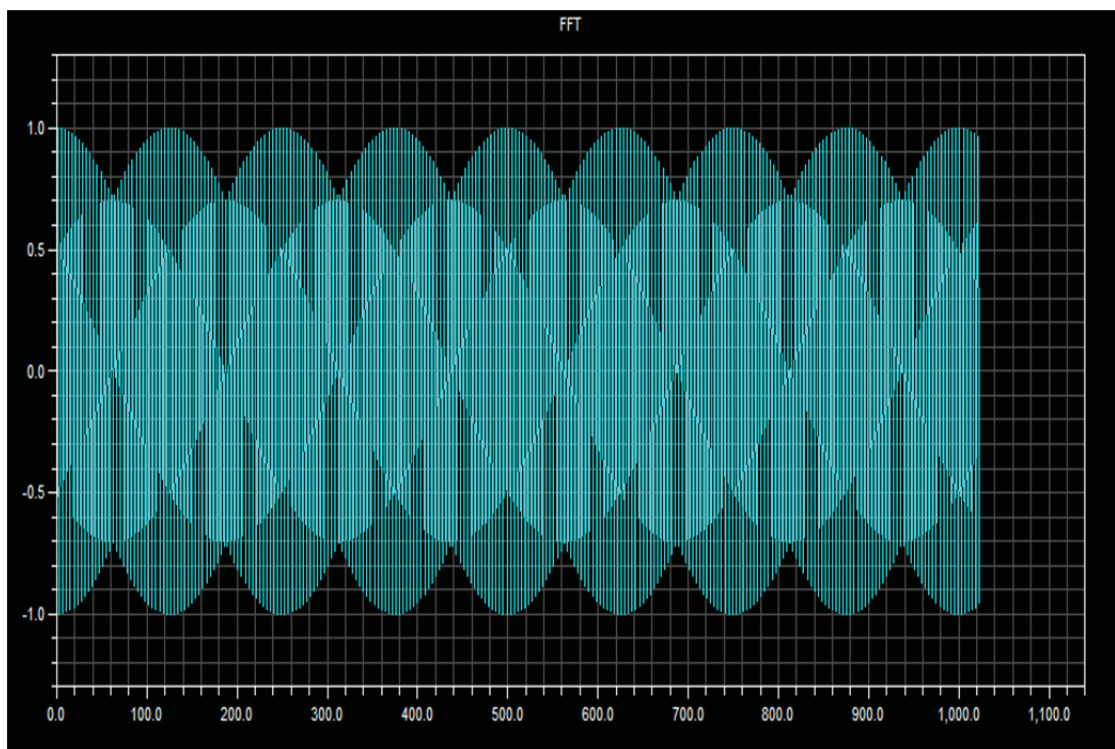


图 5.3 正弦信号实际波形

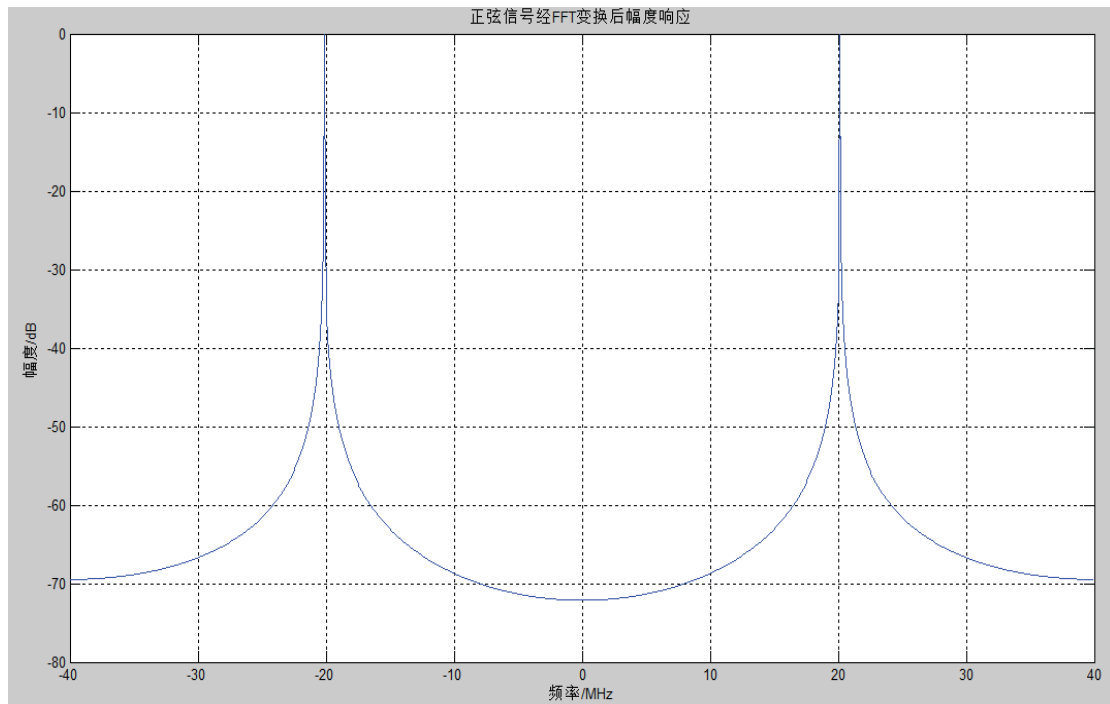


图 5.4 正弦信号经 FFT 变换后理想频谱

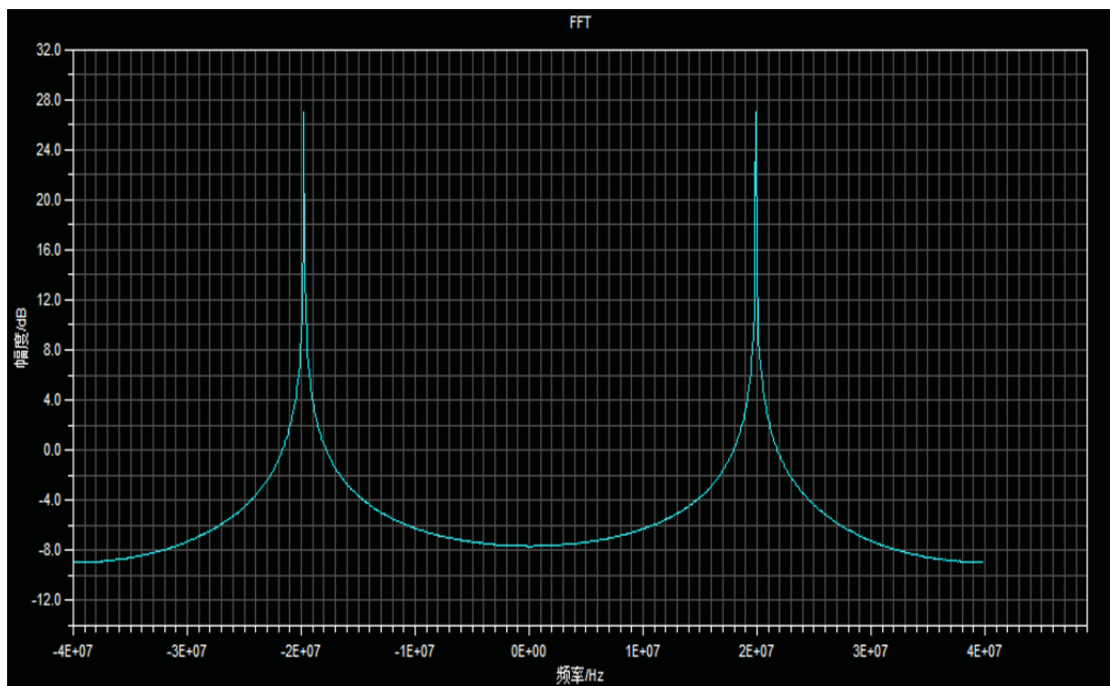


图 5.5 正弦信号经 FFT 变换后实际频谱

(2) 对应频率的复信号 $\cos(2\pi f t) + j\sin(2\pi f t)$ 的频谱特征.

a. 在 (1) 中 Matlab 程序的基础上, 产生数据文件 cos1024.dat, 求解并画出复信号经 FFT 变换后理想幅度响应。

```
x0_cos=cos(2*pi*f0*t);
y0=x0_cos+j*x0_sin;    %复信号y0
```

%%% 求复信号FFT变换并画出频域波形

```
Yw=fft(y0);  
Yw=fftshift(Yw);  
Yw=abs(Yw);  
Yw=20*log10(Yw/max(Yw));  
figure(3)  
plot(delta_1024,Yw); grid on;  
xlabel('频率/MHz');  
ylabel('幅度/dB');  
title('复信号经FFT变换后频谱响应');
```

%%% 写出cos1024.dat

```
fid = fopen('D:\dsp\FFT\FFT_spectrum\cos1024.dat','w');  
fprintf(fid,'%15.10e\n',x0_cos);
```

b. 令汇编程序中的输入实部为cos1024.dat，输入虚部为sin1024.dat

```
.VAR    redata[N]= "cos1024.dat";, .VAR    imdata[N]= "sin1024.dat";
```

c. Build Project

d. 运行程序

e. 理想频谱与实际频谱的对比如下图所示：

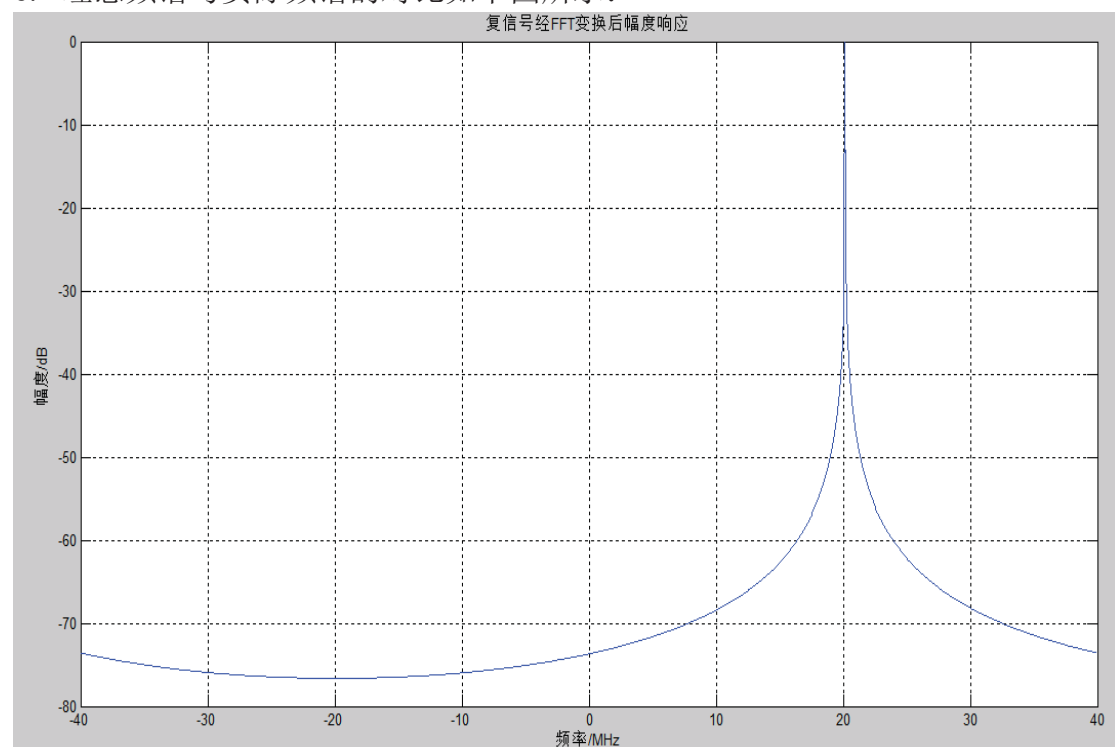


图 5.6 复信号经 FFT 变换后理想幅度响应

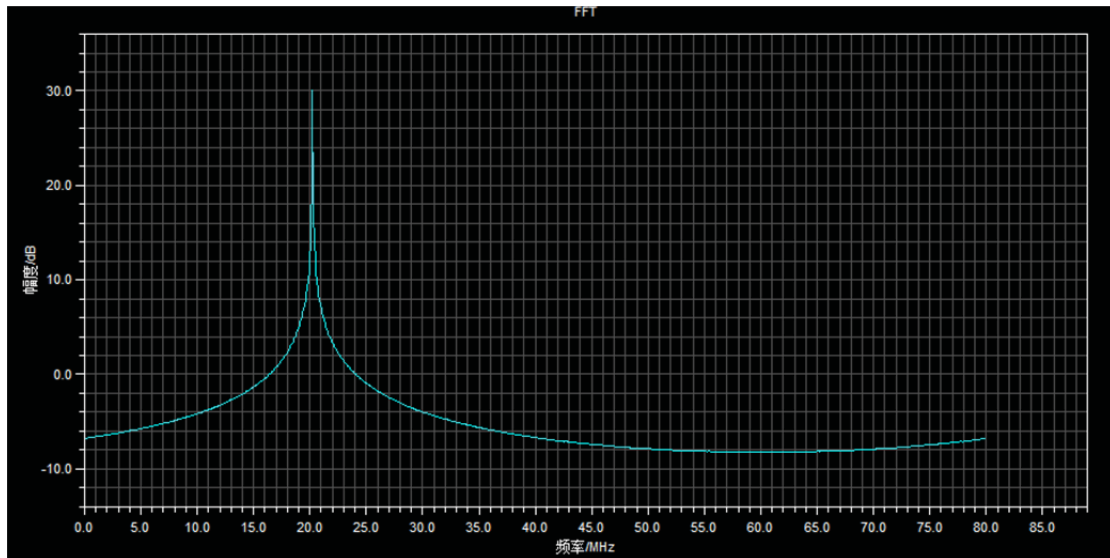


图 5.7 复信号经 FFT 变换后实际幅度响应

(3) 产生两个频率相邻的信号，观测并分析 FFT 的分辨能力。

采样率 $f_s=80\text{MHz}$ ，取 1024 个点，所以频率分辨力为 $df=80/1024=78125\text{Hz}$ 。原信号频率 f_0 为 20.16MHz ，我们在原信号基础上另加两个复信号，频率分别取 $f_1=f_0-0.5*df=20.121\text{MHz}$ ， $f_2=f_0+1.5*df=20.277\text{MHz}$ ，然后进行 FFT 变换，观察其频谱：

a. 利用 Matlab 在 (1) (2) 的基础上，产生多频率复信号，对多频率复信号进行 FFT 变换，并画出其理想频谱，并重新产生数据文件 `sin1024.dat` 及 `cos1024.dat`，程序如下：

%%% 多频率复信号的产生

```
df=fs/1024;
```

```
f1=f0-df/2
```

```
f2=f0+1.5*df
```

```
x1_sin=sin(2*pi*f1*t);
```

```
x1_cos=cos(2*pi*f1*t);
```

```
x2_sin=sin(2*pi*f2*t);
```

```
x2_cos=cos(2*pi*f2*t);
```

```
y2=(x0_cos+x1_cos+x2_cos)+j*(x0_sin+x1_sin+x2_sin);
```

%%% FFT

```
Y2w=fft(y2);
```

```
Y2w=abs(Y2w);
```

```
Y2w=20*log10(Y2w/max(Y2w));
```

```
Y2w=fftshift(Y2w);
```

%%% 画出频谱

```
figure(4)
```

```
plot(delta_1024,Y2w); grid on;
```

```
axis([-40 40 -70 3]);
```

```
xlabel('频率/MHz');
```

```
ylabel('幅度/dB');
```

```
title('多频率复信号经FFT变换后频谱1024');
```

```

%%% 重新写出数据文件sin1024.dat, cos1024.dat
fid = fopen('D:\dsp\FFT\FFT_spectrum\sine1024.dat','w');
fprintf(fid,'%15.10e\n',(x0_sin+x1_sin+x2_sin));
fid = fopen('D:\dsp\FFT\FFT_spectrum\cos1024.dat','w');
fprintf(fid,'%15.10e\n',x0_cos+x1_cos+x2_cos);

```

c. Build Project

d. 运行程序 FFT_spectrum. dpj

e. 多频率复信号在 20MHz 附近的理想与实际幅度响应如下图所示：

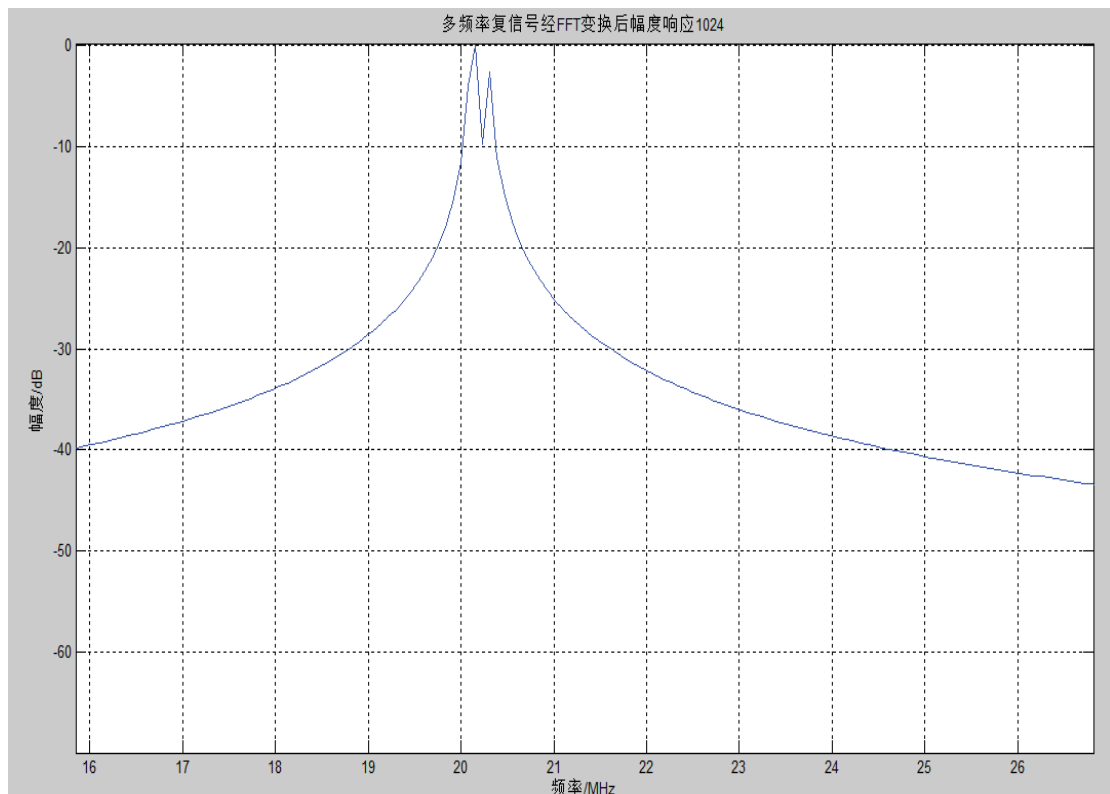


图 5.8 多频率复信号频谱在峰值附近理想频谱

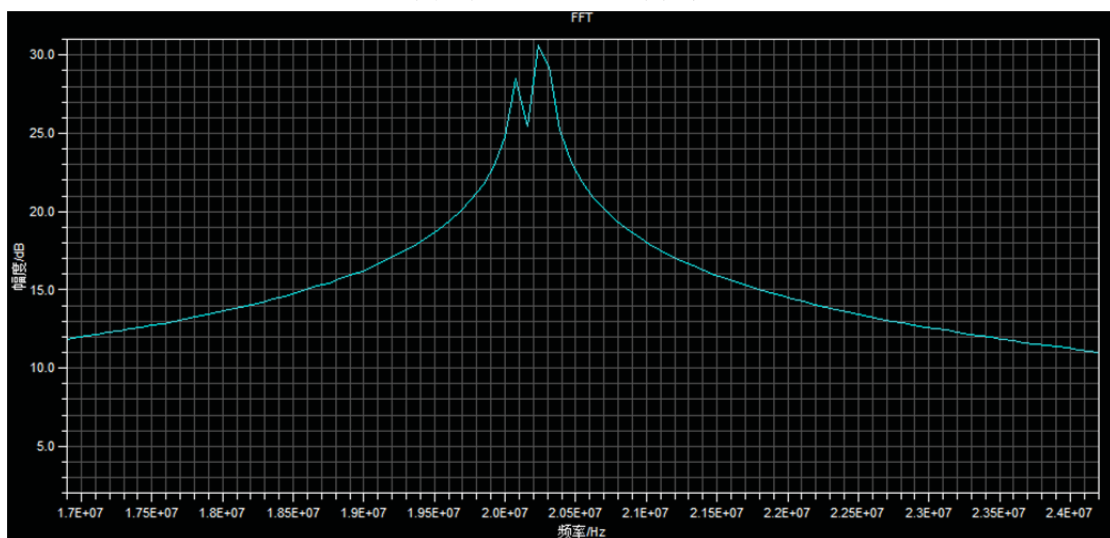


图 5.9 多频率复信号在峰值附近的实际频谱

可以看出，FFT 可以分辨出频率分别为 f 与 $f_2=f+1.5*df$ 的两个信号，但无法分辨出 f 与 $f_1=f-0.5*df$ 两个信号。可以验证 FFT 的分辨率为 $80M/1024$ ，即分辨率=采样率/所取点数；

